TECHNICAL REPORT

Inferno: Adaptive LLM Inference Serving, Measured End to End

A rigorous experimental study spanning a hand-written CUDA kernel, an MPI request router and a load-aware routing controller on a single 8 GB GPU

HARDWARE	NVIDIA GeForce RTX 4060 Laptop GPU (8 GB, sm_89, 24 SMs), 566.07 driver; Windows 11 with WSL2 (Ubuntu)
SOFTWARE	vLLM 0.11.0, PyTorch 2.8.0 (CUDA 12.8 build), CUDA toolkit 12.4, Open MPI 5.0.10, Python 3.12
MODEL	Qwen2.5-1.5B-Instruct in fp16, GPTQ-Int8 and AWQ-Int4 (official releases)
SCOPE	Problem statement, objectives, literature review, prior work, complete methodology, results and discussion
STATUS	All numbers in this report are generated from raw measurement files by scripts in the repository. Negative results are reported as such.
AUTHOR	Project owner. Report prepared September 2026.

Abstract

Serving large language models (LLMs) efficiently is a systems problem that spans GPU kernels, batching and scheduling, quantization, multi-replica routing and the statistics used to judge any of it. This report documents **Inferno**, an experimental testbed that studies all of these layers together on one 8 GB laptop GPU, and asks a single question: *can an adaptive routing policy over pre-loaded, heterogeneous replicas beat every fixed configuration under realistic, bursty traffic, and where do the gains or losses come from?*

The study makes six measurements. (1) A hand-written fused int4 dequantize-and-matrix-vector CUDA kernel, verified against a double-precision CPU reference before any timing, reaches 95% of theoretical DRAM bandwidth (97% of the measured streaming-read bandwidth) on the best layer and 3.67× the speed of fp16 cuBLAS, against a byte-ratio ceiling of 3.76×; the bandwidth figures were corrected during the work after a benchmark flaw (an L2 leak of about 6%) was found. (2) Patched into vLLM 0.11 as a torch custom op and verified on the real checkpoint, the kernel decodes Qwen2.5-1.5B at 144 tokens/s with one sequence, against 143 for vLLM's default Marlin path (1.01×), 136 for its classic AWQ kernel and 67 for fp16: parity with Marlin, not a win, and with four concurrent sequences Marlin is faster (0.68×). An earlier claim that the kernel beat Marlin was a measurement error and is retracted. (3) In a 96-run paired campaign (four policies, three load patterns, eight seeded repetitions) the pre-specified hypothesis that an adaptive router beats the strongest fixed configuration is **rejected**: adaptive p99 latency was +112% to +204% relative to the best fixed baseline (Holm-corrected $p = 0.023$, the smallest attainable at $n = 8$). (4) A controlled follow-up isolates the cause: a second *active* replica on the same GPU multiplies p99 by 2.06× and 2.08× at 30% and 60% of capacity, while a resident but idle replica costs +0.8%; a headroom analysis of the existing data shows that no pattern-aware choice between the two fixed tiers could have gained more than 3.3% of p99. (5) An MPI request router adds about 2 µs per hop in isolation and is statistically indistinguishable from an HTTP router end to end. (6) Timed against Marlin, classic AWQ and fp16 on every decode layer at batch 1 to 64, the kernel clearly wins only layers worth 15% of decode GEMM time, at batch 1, and is up to 7.4× slower above that, so by a pre-stated rule Marlin stays the serving default. The same benchmark’s reference check found Marlin returning wrong results (relative error up to 0.28) at batch 32, 64, 128 on this machine, a finding not yet reproduced elsewhere.

Beyond the results, the report documents the measurement methodology in detail: open-loop load, capacity-relative load calibration, paired seeded workloads with randomized execution order, correctness gates before timing, L2-defeating kernel benchmarking, exact non-parametric tests with multiple-comparison control, and an audit trail of the errors found and fixed along the way, including two that overturned earlier claims. It states plainly what the evidence does not show, chiefly that the adaptive idea remains untested where each replica has its own GPU.

Keywords: LLM inference serving, CUDA, int4 quantization, memory-bound kernels, MPI, adaptive routing, GPU co-residency, benchmarking methodology, paired statistics.

Contents

Abstract 2

1 Introduction 7

1.1 Context and motivation 7

1.2 Why a single small GPU 7

1.3 Contributions 8

1.4 How to read this report 8

2 Problem statement 9

2.1 The engineering problem 9

2.2 Formalization 9

2.3 Why it is hard 10

2.4 Scope, assumptions and non-goals 10

3 Objectives, research questions and hypotheses 11

3.1 Overall aim 11

3.2 Research questions 11

3.3 Hypotheses with operational definitions 11

3.4 Measurable objectives and acceptance criteria 12

3.5 Falsifiability and what would change the conclusions 12

4 Literature review 13

4.1 Hardware behaviour of transformer inference 13

4.1.1 Prefill, decode and the KV cache 13

4.1.2 The roofline view 13

4.2 Batching and scheduling in LLM serving systems 14

4.3 Prefill/decode interference and disaggregation 14

4.4 Multi-instance scheduling, routing and adaptive serving 15

4.5 Quantization and quantized kernels 15

4.5.1 Algorithms 15

4.5.2 Kernels 15

4.6 Sharing a GPU between processes 16

4.7 Message passing in high-performance computing 16

4.8 Benchmarking methodology and statistics 16

4.8.1 Workload generation and measurement 16

4.8.2 Rigorous performance evaluation 17

4.8.3 Statistical tools 17

4.9 Synthesis and gap analysis . . . 17

5 Previous work and starting point . . . 18

5.1 Closest prior systems and how this work relates . . . 18
5.2 Starting point of this project . . . 18
5.3 What was retained and what changed . . . 19
5.4 Previous work in this repository . . . 19

6 System design and architecture . . . 20

6.1 Design constraints . . . 20
6.2 Overall architecture . . . 20
6.2.1 Component boundaries . . . 21
6.3 The replica pool . . . 21
6.4 The adaptive controller . . . 21
6.5 Serving harness and client . . . 22
6.6 The MPI router . . . 23
6.7 The fused int4 kernel . . . 23
6.7.1 Weight format . . . 23
6.7.2 The algebraic form . . . 24
6.7.3 Integration into vLLM . . . 24
6.7.4 The optimization ladder . . . 24

7 Methodology . . . 26

7.1 Principles . . . 26
7.2 Workload model and load calibration . . . 26
7.2.1 Arrival processes . . . 26
7.2.2 Capacity calibration . . . 27
7.3 Metric definitions . . . 28
7.4 Kernel methodology . . . 28
7.4.1 Roofline framing and the byte-ratio ceiling . . . 28
7.4.2 Correctness gate . . . 29
7.4.3 Timing discipline . . . 29
7.4.4 In-situ comparison against the vLLM kernels . . . 29
7.4.5 End-to-end decode benchmark . . . 30
7.4.6 Profiler counters . . . 30
7.5 Campaign design: adaptive versus fixed . . . 30
7.5.1 Factors, cells and repetitions . . . 30
7.5.2 Pairing and randomization . . . 30
7.5.3 Choice of baseline . . . 31
7.5.4 Statistical protocol . . . 31

7.5.5 Why eight repetitions . . . 31
7.5.6 Decision rule for H1 . . . 32
7.6 Mechanism study: co-residency contention . . . 32
7.7 MPI router methodology . . . 32
7.7.1 Microbenchmarks . . . 32
7.7.2 Clock argument and per-hop timestamps . . . 33
7.7.3 Scaling and router comparison . . . 33
7.8 Configuration sweep and drain handling . . . 33
7.9 Threats to validity . . . 34
7.10 Audit trail: errors found by running the protocol . . . 34

8 Implementation and instrument verification . . . 36
8.1 Code base . . . 36
8.2 Reproducibility of the environment . . . 36
8.3 Test suite . . . 36
8.4 Instrument verification steps performed . . . 37

9 Results . . . 38
9.1 Kernel study . . . 38
9.1.1 Correctness . . . 38
9.1.2 Measurement corrections . . . 38
9.1.3 Timing by version . . . 38
9.1.4 What each optimization stage bought . . . 40
9.1.5 Small batches . . . 41
9.2 Comparison with vLLM’s kernels, end-to-end decode and server capacity . . . 41
9.2.1 Kernel time in situ . . . 41
9.2.2 End-to-end decode throughput . . . 42
9.2.3 Server-level capacity . . . 43
9.3 Where the custom kernel earns its place: every layer, batch 1 to 64 . . . 43
9.4 Correctness probe: Marlin returned wrong results at some batch sizes . . . 45
9.5 Campaign: adaptive versus fixed routing . . . 46
9.5.1 Headroom analysis . . . 47
9.5.2 Energy and throughput . . . 48
9.6 Mechanism study: two active servers on one GPU . . . 48
9.7 MPI routing . . . 50
9.8 Scaling and MPI versus HTTP with real replicas . . . 50
9.9 Quantization and batch-cap sweep . . . 51

10 Discussion . . . 52
10.1 Answers to the research questions . . . 52

10.2 Status of the four hypotheses . . . 52
10.3 Why the adaptive policy lost . . . 53
10.4 Why the kernel result should not be over-read . . . 53
10.5 Methodological observations . . . 54

11 Limitations and future work . . . 55
11.1 Limitations . . . 55
11.2 Future work, in order of expected value . . . 56

12 Conclusion . . . 57

Appendix A. Reproducing the study . . . 58

Appendix B. Configuration . . . 59

Appendix C. Glossary . . . 60

Appendix D. Provenance of results . . . 61

References . . . 62

1 Introduction

1.1 Context and motivation

Transformer-based language models [1] generate text one token at a time. Each request therefore has two phases with very different hardware behaviour: a *prefill* phase that processes the whole prompt in parallel and is compute-intensive, and a *decode* phase that produces one token per step and is memory-intensive [2]. In decode, every step re-reads all model weights and the growing key-value (KV) cache from DRAM to perform, per weight, roughly one multiply-add. Runtime is then set by bytes moved rather than arithmetic, the regime described by the roofline model [3]. Two consequences dominate the design of serving systems: batching many requests raises arithmetic intensity and throughput, and shrinking the bytes per weight through quantization [4,5] can speed decode up by nearly the compression ratio if the kernels are good enough [6].

Production engines such as vLLM [7] implement continuous batching in the style of Orca [8], paged KV-cache management, chunked prefill [9] and quantized kernels. Around them sits a second layer of decisions that is much less standardized: how many replicas to run, how each is configured (batch cap, prefill chunk size, precision), and how requests are routed among them as load shifts [10,11,12]. This layer is where the question of *adaptivity* lives: a configuration that is best at low load is rarely best under a burst, and reconfiguring a running server is expensive.

1.2 Why a single small GPU

This study deliberately uses one consumer GPU (an RTX 4060 Laptop with 8 GB) and a 1.5-billion-parameter model. That choice is a constraint, not a preference: an 8-billion-parameter model in fp16 needs about 16 GB and cannot run here. It has an upside that shapes the whole report. Decode on this GPU is memory-bound in exactly the way it is on data-centre GPUs, the experiments are cheap enough to repeat many times, and every effect can be measured with paired designs and confidence intervals rather than single runs. It also has a cost, which the report treats as a first-class limitation: replicas that share one GPU contend with each other, which turned out to be the single most important finding (Chapter 9).

1.3 Contributions

1. **A verified fused int4 GEMV kernel** with a seven-stage optimization ladder that reaches the memory limit on large layers, a correctness gate that caught a real bug, an L2-defeating benchmark harness with run-to-run spread, a small-batch sweep that locates where a CUDA-core kernel stops paying off, and an integration into vLLM with an end-to-end comparison against the kernels vLLM actually runs (parity with the default Marlin path at one sequence).
2. **A capacity-relative, open-loop, paired benchmarking methodology** for LLM serving, including load calibration, seeded workloads, randomized execution order, drain-aware throughput and energy accounting.
3. **A pre-specified test of adaptive routing** with a 96-run campaign, the strongest fixed baselines, and generated verdicts that state null and negative results in plain language: the hypothesis was rejected.
4. **A controlled mechanism study** that explains the negative result (GPU co-residency contention) rather than merely reporting it.
5. **An MPI request router with per-hop instrumentation**, GPU-free coordination microbenchmarks, and a real-replica comparison against an HTTP router.
6. **An audit trail** of measurement errors found and corrected during the work, including a benchmark flaw that inflated bandwidth figures and a baseline measurement that had wrongly made the custom kernel look faster than Marlin; offered as methodology in its own right.

1.4 How to read this report

Chapters 2 and 3 state the problem and the falsifiable objectives. Chapters 4 and 5 review the literature and the state of prior work. Chapters 6 and 7 give the system design and the complete methodology, and are written to be sufficient for replication. Chapters 8 to 10 report results, discussion and limitations. Appendices contain reproduction commands, configurations, full data tables and a glossary. References marked with a dagger were checked against the publisher or preprint page while preparing this report; others are cited from the author's knowledge and should be confirmed before formal submission.

2 Problem statement

2.1 The engineering problem

A serving operator runs one or more model replicas on GPUs and receives a stream of requests whose arrival rate and length distribution change over time. Each request carries latency expectations: the first token should appear quickly (time to first token, TTFT) and subsequent tokens should stream steadily (time per output token, TPOT). The operator chooses a *configuration* for each replica (numeric precision, maximum concurrent sequences, prefill chunk size) and a *routing rule* that assigns each request to a replica. The question is whether a routing rule that reacts to observed load can outperform the best single static configuration, and by how much.

2.2 Formalization

Let a request i arrive at time a_i with prompt length p_i and output length o_i. A policy π maps the request and the observable system state s (in-flight counts per replica, recent latency statistics) to a replica index. Under policy π the request experiences a first-token time and a completion time, giving TTFT_i and TPOT_i. With latency objectives τ_{TTFT} and τ_{TPOT}, *goodput* is the rate of requests meeting both [13]:

$$\mathrm{Goodput}(\pi) = \frac{1}{T}\sum_{i} \mathbf{1}[\mathrm{TTFT}_i \le \tau_{\mathrm{TTFT}} \ \wedge \ \mathrm{TPOT}_i \le \tau_{\mathrm{TPOT}}] \tag{1}$$

The comparison of interest is between an adaptive policy π_{A} and the best of a set of fixed policies $F = \{\pi_1, \ldots, \pi_m\}$, evaluated on the tail of the latency distribution, because operators are held to tail behaviour [14]:

$$\Delta_p = \frac{p_{99}(\pi_A) - \min_{\pi \in F} p_{99}(\pi)}{\min_{\pi \in F} p_{99}(\pi)} \qquad H_1 : \ \Delta_p < 0 \ \text{ at non-inferior throughput} \tag{2}$$

Two properties make this hard to answer credibly. First, the value of adaptivity depends entirely on how strong the fixed baselines are: a policy that beats a badly chosen baseline proves nothing. Second, the answer depends on hardware facts that papers rarely isolate, such as what happens when two replicas share a device.

2.3 Why it is hard

- **Reconfiguration is expensive.** vLLM cannot change its batch cap or numeric precision on a live server; that requires a restart, which this study measures at roughly 48 s, several times the duration of a traffic burst. Adaptivity must therefore act by routing across pre-loaded configurations.
- **Heterogeneity cuts both ways.** A low-latency configuration and a high-throughput configuration each win in some regimes, but co-locating them has a cost.
- **Tail metrics need many samples and paired designs.** With few repetitions, run-to-run variation on a thermally-limited laptop GPU can be larger than the effect being studied.
- **Measurement artefacts are easy to create.** Closed-loop load generators hide queueing [15]; caches make small kernels look faster than the memory system allows; wall-clock throughput that ignores backlog drain makes every policy look alike. Each of these was encountered and is addressed in Chapter 7.

2.4 Scope, assumptions and non-goals

Table 1. Scope of the study.

In scope	Out of scope
Batch-1 decode kernel (int4 weights, fp16 activations, group size 128)	Tensor-core kernels for large batches; training; attention kernels
Routing across pre-loaded replicas on one GPU; HTTP and MPI routers	Multi-GPU tensor or pipeline parallelism; live KV-cache migration
Latency (TTFT, TPOT, e2e), throughput, goodput, energy per token	Model quality: checkpoint accuracy after quantization is not evaluated
Synthetic prompt text with server-verified token counts	Real user prompts and semantic effects (e.g. prefix caching)
One model family (Qwen2.5-1.5B) in fp16 / int8 / int4	Larger models; the 7B cloud configuration is written but not run

Assumptions. (i) The workload is a stationary or piecewise-stationary Poisson process whose parameters are known to the experimenter (not to the policy). (ii) Output length is controlled by the client and identical across configurations (the `ignore_eos` flag is set), so throughput comparisons are not confounded by different generation lengths. (iii) All processes run on one host, so a single monotonic clock is shared across processes (used to timestamp MPI hops).

3 Objectives, research questions and hypotheses

3.1 Overall aim

To determine, with defensible statistics and honest reporting, whether and why load-aware routing over heterogeneous replicas improves LLM serving on a single GPU; and to characterize the memory-bound kernel, the multi-replica communication layer and the energy/latency trade-offs that any such policy sits on top of.

3.2 Research questions

Table 2. Research questions.

ID	Research question
RQ1	How close can a hand-written fused int4 GEMV get to the memory roofline, and how does it compare with the kernels vLLM runs?
RQ2	Does an adaptive routing policy achieve lower p99 latency than the strongest fixed configuration at non-inferior throughput under steady, bursty and mixed-length load?
RQ3	If not, what is the mechanism? In particular, what does it cost to run a second active replica on the same GPU?
RQ4	What does MPI-based request routing cost, in isolation and inside a real serving path, and how does it scale with replicas?
RQ5	How do quantization level and batch cap trade latency, throughput and energy, and what does reconfiguration cost?

3.3 Hypotheses with operational definitions

Hypotheses were fixed, together with the analysis code and decision rules, before the campaign was run. Each has a stated boundary condition so that a null or negative result is informative.

Table 3. Hypotheses, tests and boundary conditions.

ID	Statement	Operational test	Boundary condition
H1	Adaptive routing lowers p99 versus the best fixed configuration at non-inferior throughput.	Per load pattern: paired (by repetition) difference in p99 against the fixed policy with the lowest mean p99 among error-free policies; significant only if the paired t-test and the Wilcoxon signed-rank test agree at Holm-adjusted $\alpha = 0.05$; throughput $\geq 98\%$ of the baseline.	Under perfectly steady load a well-tuned fixed configuration is expected to match adaptive.
H2	MPI multi-replica routing scales sub-linearly and its coordination overhead is measurable.	Scaling efficiency = throughput(N) / (N × throughput(1)); per-hop overhead from inter-rank timestamps; MPI vs HTTP paired comparison.	Replicas share one GPU, so linear scaling is not expected.
H3	A hand-written fused int4 kernel reaches a fraction of the library kernel's throughput, and the gap is explained by profiler counters.	Median-of-invocations kernel time vs cuBLAS fp16 and in-situ vLLM kernels; counter-level explanation via Nsight Compute.	Batch 1 and very small batches; counters need GPU counter access.
H4	Systems-level adaptivity, not model changes, drives efficiency gains (tokens per joule).	Adaptive tokens/joule relative to the best fixed configuration, per load pattern.	Energy sampled from the GPU only, at 200 ms resolution.

3.4 Measurable objectives and acceptance criteria

Table 4. Objectives and acceptance criteria.

#	Objective	Acceptance criterion
O1	Reproducible environment	A clean checkout builds and runs the smoke test; environment doctor passes; resolution date-pinned.
O2	Trustworthy measurement harness	Open-loop client, TTFT/TPOT/e2e/goodput, power and energy; unit-tested against mock servers.
O3	Correct, fast kernel	All kernel variants match a CPU reference on every shape in every invocation; timing with L2 defeated and spread reported.
O4	Capacity-relative load	Request rates derived from measured saturation capacity, not guessed.
O5	Adequately powered H1 test	≥ 8 paired repetitions (7 is the theoretical minimum at which a Holm-corrected Wilcoxon test over three patterns can fall below 0.05, with no margin; 8 leaves headroom).
O6	Mechanism isolation	A controlled experiment that varies only how the GPU is shared, at fixed total load.
O7	MPI characterization	GPU-free microbenchmarks plus a paired real-replica comparison with an HTTP router.
O8	Generated, auditable reporting	Every number in reports traces to a raw file; verdict text produced by code.

3.5 Falsifiability and what would change the conclusions

H1 would have been supported had the adaptive policy shown a significantly lower p99 than the best fixed configuration on at least one non-steady pattern with non-inferior throughput. It was not. The conclusions would change if (i) the adaptive policy were re-evaluated with replicas on separate GPUs and won, (ii) a stronger fixed baseline were found that the campaign excluded, or (iii) the measurement instruments were shown to be biased. Chapter 7 describes the checks that guard against (iii); Chapter 11 lists what remains open for (i).

4 Literature review

The review is organized by the layers this study touches: the hardware behaviour of LLM inference, serving-system scheduling, prefill/decode interference, multi-instance routing, quantization and its kernels, GPU sharing, message passing in high-performance computing, and benchmarking and statistical practice. Each section ends with what it implies for the design of this study. A synthesis and gap analysis closes the chapter.

4.1 Hardware behaviour of transformer inference

4.1.1 Prefill, decode and the KV cache

Autoregressive generation with the Transformer architecture [1] caches the keys and values of previous tokens so each new token attends over stored state instead of recomputing it. A request then divides into a prompt-processing phase and a token-generation phase. Characterizing production-like workloads, the Splitwise study reports that the prompt phase is compute-intensive while the token-generation phase is memory-intensive, with distinct latency, throughput, memory and power profiles [2]. This asymmetry is the root of most serving-system design and of the heterogeneous-tier design used later in this report.

4.1.2 The roofline view

The roofline model [3] bounds attainable performance by the minimum of a compute roof and a memory roof, and identifies a kernel's regime by its *arithmetic intensity*: floating-point operations per byte moved from DRAM.

$$P_{\text{attainable}} = \min\left(P_{\text{peak}},\ B_{\text{DRAM}} \cdot I\right), \qquad I = \frac{\text{FLOPs}}{\text{bytes moved}} \tag{3}$$

For a matrix-vector product with an $N \times K$ weight matrix in fp16, FLOPs = $2NK$ and bytes $\approx 2NK$, so $I \approx 1$ FLOP/byte, far to the left of any modern GPU's ridge point: decode at batch size 1 is memory-bound (figure below). Pope et al. analyse this trade-off when partitioning large Transformer inference across accelerators and show how batch size and memory bandwidth govern cost and latency [16]. FlashAttention [17,18] is the canonical example of the same principle applied to attention: reduce traffic between GPU high-bandwidth memory and on-chip SRAM rather than reduce arithmetic.

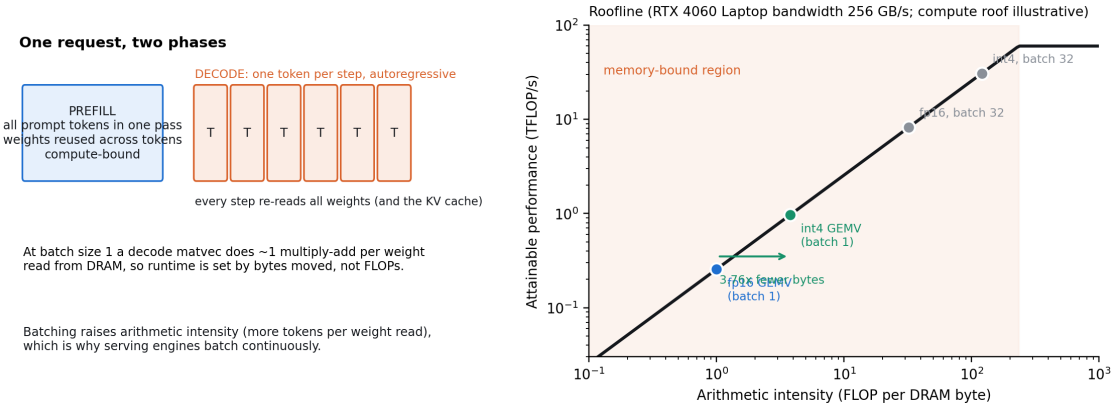


Figure 1. Roofline view of prefill and decode: decode at small batch sits on the memory roof, prefill near the compute roof.

> **Implication for this study**
> The kernel study (Chapter 9) therefore uses achieved DRAM bandwidth, and speedup against the *byte-ratio ceiling*, as the figures of merit, and the serving study treats batching as the mechanism that moves decode toward the compute roof.

4.2 Batching and scheduling in LLM serving systems

Early inference servers batched at request granularity, so a batch waited for its longest sequence. **Orca** [8] introduced *iteration-level scheduling*: the scheduler invokes the execution engine for a single model iteration at a time, so finished requests leave and new ones join between iterations; it pairs this with *selective batching*, batching only the operations that allow it. On GPT-3 175B the authors report a large throughput improvement over NVIDIA FasterTransformer at comparable latency. This *continuous batching* is now standard.

vLLM [7] attacks the memory side. KV-cache memory per request is large and grows and shrinks dynamically, so contiguous allocation wastes memory through fragmentation and duplication and limits batch size. PagedAttention stores the KV cache in fixed-size blocks mapped to non-contiguous physical memory, borrowing virtual-memory paging, and reports 2-4× higher throughput than prior systems at similar latency. The engine used throughout this study is vLLM [19].

Sarathi-Serve [9] addresses the throughput-latency tension inside a single replica. Long prompts, if prefilled in one iteration, stall the decode of every other request in the batch. *Chunked prefill* splits a prompt into near-equal chunks and builds *stall-free* schedules that add new work without pausing ongoing decodes. The bound on tokens per iteration (in vLLM, `max_num_batched_tokens`) becomes a control knob trading TTFT of long prompts against decode smoothness for everyone else. This is the knob that differentiates the two replica tiers in this study.

Orthogonal techniques reduce work per token rather than schedule it: speculative decoding [20] lets a cheap draft model propose tokens that the target model verifies in parallel; SGLang [21] adds a runtime for structured multi-call programs with prefix-cache reuse. Neither is used here; both are complementary to the routing question.

4.3 Prefill/decode interference and disaggregation

DistServe [13] observes that colocating prefill and decode across all requests causes strong prefill-decode interference and couples the resource allocation and parallelism plans of both phases. It assigns the phases to different GPUs, co-optimizes each phase's parallelism for TTFT and TPOT targets, and optimizes *goodput*, the rate of requests that meet the latency objectives; the authors report serving several times more requests, or meeting much tighter latency objectives, than the state of the art. **Splitwise** [2] reaches a similar architecture from a characterization and power/cost perspective, splitting the phases across machines so each can run on hardware suited to it.

Two observations matter here. First, both systems put the specialized phases on *separate devices*; their premise is that co-locating heterogeneous work on one device creates interference. Second, the goodput metric they popularized is adopted in this study. The heterogeneous-tier design used here (a small-chunk decode-oriented replica and a large-chunk throughput-oriented replica) is a lightweight relative of phase specialization; but because the two tiers share one GPU, the interference premise applies to the *replicas themselves*, which the

results confirm.

4.4 Multi-instance scheduling, routing and adaptive serving

General model-serving systems introduced several ideas this study revisits. **Clipper** [11] interposes between applications and models and uses caching, adaptive batching and bandit-based model selection to control latency, throughput and accuracy. **INFaaS** [22] lets users state performance and accuracy goals and selects among model variants and hardware to meet them, autoscaling as load changes. **AlpaServe** [12] shows that model parallelism can be used for statistical multiplexing across devices under bursty load, exposing a trade-off between parallelism overhead and multiplexing gains.

For LLMs specifically, **Llumnix** [10] reschedules requests across model instances at run time, using live migration of requests and their in-memory state, to improve load balancing and isolation and to differentiate priorities; it reports order-of-magnitude tail-latency improvements against strong baselines. Its adaptivity acts by *moving work between instances*; this study's adaptivity acts by *choosing the instance for new work*, a cheaper and weaker mechanism appropriate to what a stock vLLM server exposes.

A pattern across these systems is that instances are assumed to have independent accelerators. None of the works reviewed here isolates what happens when independent inference servers share one device, which is the question Section 9.4 answers. This is a statement about the set of papers reviewed, not a claim that no such study exists.

4.5 Quantization and quantized kernels

4.5.1 Algorithms

Weight-only post-training quantization compresses weights to 4 or 8 bits while activations stay in higher precision. **LLM.int8()** [23] handles outlier features with mixed-precision decomposition; **SmoothQuant** [24] migrates activation outliers into weights to enable 8-bit weight-and-activation quantization; **GPTQ** [4] quantizes one-shot using approximate second-order information, reaching 3-4 bits per weight with small accuracy loss; **AWQ** [5] protects salient weight channels by observing activation statistics, avoiding backpropagation and reconstruction, and received the MLSys 2024 best paper award. Group-wise asymmetric quantization, in which each group of consecutive weights (typically 128) shares a scale and a zero point, is the format targeted by the kernel in this study.

4.5.2 Kernels

A quantized model is only faster if its kernels avoid materializing dequantized weights in DRAM. **MARLIN** [6] is a family of mixed-precision linear kernels for 4-bit weights; the authors show that batch sizes up to 16-32 can be supported with close to the maximum (nominally 4×) quantization speedup, with gradually decreasing benefit up to 64-128, using asynchronous memory access, task scheduling and pipelining, and report up to 2.8× end-to-end speedup inside vLLM. The 'nominal 4×' refers to the weight payload; this report shows that with group-128 fp16 scales and zero points the byte-ratio ceiling for a matrix-vector product is closer to 3.76× (Section 7.4). vLLM ships several quantized paths [19]: the original AWQ GEMM kernel (selected by `quantization=awq`) and Marlin-based paths (`awq_marlin`, `gptq_marlin`). Marlin is designed for parallel (batched) clients; this study measures them in situ, end to end and at saturated serving: on this GPU Marlin is at the bandwidth limit at batch 1 and clearly faster with concurrent sequences, while the classic AWQ kernel is slower in both regimes.

> **Implication for this study**
> A kernel comparison is regime-dependent. Any claim about a quantized kernel must state the batch size, the layer shapes, and whether the measurement is isolated or in situ. This report does all three.

4.6 Sharing a GPU between processes

NVIDIA's documentation describes the options for sharing a device among processes [25]. By default, work from different processes is time-sliced on the GPU; the Multi-Process Service (MPS) can allow kernels from multiple processes to run concurrently, and Multi-Instance GPU (MIG) partitions supported data-centre GPUs into isolated instances. The consumer laptop GPU and WSL2 environment used here provide default time-slicing; MPS and MIG were not evaluated (MIG is unavailable on this class of GPU, and MPS support under WSL2 was not explored). Time-slicing implies that two active servers each see a fraction of GPU time, pay context-switch overhead, and, because each holds only part of the request stream, run smaller batches than one merged server would. Section 9.4 measures this effect directly.

4.7 Message passing in high-performance computing

MPI [26] is the standard message-passing interface of HPC; Open MPI [27] is a widely used implementation. Performance of message passing is commonly reasoned about with latency-bandwidth (alpha-beta) models and, more richly, with LogP [28], whose parameters capture latency, overhead, gap and processor count. MPI implementations typically switch between an *eager* protocol for small messages and a *rendezvous* protocol for large ones, producing a latency step at a size threshold; the step observed in Chapter 9 between 32 KB and 256 KB is consistent with such a switch, though the exact threshold was not determined.

MPI is rarely used for the control plane of LLM serving: production systems use HTTP or gRPC front ends and collective libraries for tensor-parallel communication. It is used here because the HPC-oriented objective of the project calls for an MPI program, and because that choice is informative: an honest evaluation needs an HTTP baseline, which this study includes.

4.8 Benchmarking methodology and statistics

4.8.1 Workload generation and measurement

Schroeder, Wierman and Harchol-Balter show that *open* workload generators (arrivals independent of completions) and *closed* ones (arrivals triggered by completions) can lead to very different conclusions about scheduling policies, and that the choice must match the system being modelled [15]. A closed-loop client silently reduces offered load as latency rises, hiding queueing. Dean and Barroso emphasise that tail latency, not the mean, governs user-visible quality in large services [14]. Little's law [29] ($L = \lambda W$) connects arrival rate, latency and in-flight concurrency and is used here to reason about which batch caps are ever active.

4.8.2 Rigorous performance evaluation

A line of work argues that performance claims must be statistically defensible. Georges, Buytaert and Eeckhout show how non-determinism makes single-run comparisons misleading and advocate confidence intervals and appropriate tests [30]. Hoefler and Belli survey practice in parallel computing, find that it is often unclear whether reported improvements are deterministic or observed by chance, and give twelve guidelines for experimental design and reporting [31]. Mytkowicz et al. demonstrate that innocuous setup choices can bias results substantially [32], and Kalibera and Jones treat how many repetitions and at what levels are needed [33]. MLPerf Inference [34] standardizes inference benchmarking, including latency-bound scenarios. These works motivate the paired designs, repetition counts, spread reporting and correctness gates used here.

4.8.3 Statistical tools

The comparison uses the paired Student t-test [35] and the Wilcoxon signed-rank test [36], which is distribution-free and exact for small samples; Holm's step-down procedure [37] to control the family-wise error rate across load patterns; Cohen's paired effect size d_z [38]; and the bootstrap [39] for confidence intervals of paired differences. A property of the exact Wilcoxon test drives the study design: with n pairs all in one direction, the smallest attainable two-sided p is $2/2^n$, which is 0.0625 for $n = 5$ and 0.0078 for $n = 8$.

4.9 Synthesis and gap analysis

Table 5. Gap analysis.

Dimension	State of the literature	Gap addressed by this study
Kernel comparison	Marlin reports near-maximal speedup for batches 16-32 and beyond; AWQ kernels widely used.	Batch-1 and small-batch comparison on a small GPU with a byte-exact ceiling and a measured bandwidth reference; in-situ and end-to-end (vLLM integration) results; regime dependence made explicit; correctness gate and spread reported.
Adaptive routing	Llumnix, Clipper, INFaaS, AlpaServe adapt via migration, model selection or placement; instances assumed on separate accelerators.	Pre-specified, paired test of routing over pre-loaded heterogeneous replicas against the strongest fixed baselines in the same pool; negative result with mechanism.
GPU co-residency	Interference studied inside a batch (prefill vs decode); disaggregation puts phases on separate devices.	Controlled measurement of active vs idle second replica at fixed total load.
MPI in serving	MPI standard in HPC; not standard for LLM control planes.	Per-hop instrumented MPI router vs HTTP baseline, isolated and in situ.
Methodology	Strong general guidance (open loop, tails, paired stats) but often not applied in LLM-serving evaluations.	End-to-end application: capacity-relative load, seeded pairing, randomized order, drain-aware metrics, generated verdicts, error audit trail.

5 Previous work and starting point

5.1 Closest prior systems and how this work relates

The table compares the systems most closely related to the questions asked here. The purpose is positioning, not ranking: the systems solve different problems at different scales, on data-centre hardware, and none was re-run in this study.

Table 6. Related systems and relation to this work.

System	What it decides	Where adaptivity acts	Accelerator assumption	Relation to this study
Orca [8]	Which requests run each iteration	Iteration-level scheduling	One model instance (multi-GPU for large models)	Continuous batching is what vLLM already provides here; not re-studied.
vLLM [7]	KV-cache block allocation, batching	Memory management, scheduler	Single or multi-GPU per engine	The serving engine used throughout; its knobs define the replica tiers.
Sarathi-Serve [9]	How prefills are chunked	Per-iteration token budget	Per replica	The prefill chunk size distinguishes tier 0 (512) from tier 1 (4096).
DistServe [13]	Which GPUs run prefill vs decode	Static placement optimized for goodput	Separate GPUs per phase	Goodput metric adopted; premise that co-location interferes is tested for replicas.
Splitwise [2]	Which machines run which phase	Phase splitting with cluster provisioning	Separate machines per phase	Characterization motivates decode being memory-bound.
Llumnix [10]	Where running requests live	Live migration across instances	Independent instances	Adaptivity by migration; here only by placement of new work.
Clipper / INFaaS [11,22]	Which model variant serves a query	Adaptive batching, bandit / SLO-driven selection	Cluster of serving nodes	Ancestors of routing among configurations; not LLM-specific.
AlpaServe [12]	Model placement and parallelism	Statistical multiplexing under bursts	Cluster of GPUs	Motivates bursty load patterns; multiplexing needs separate devices.
MARLIN [6]	How 4-bit GEMM executes	Kernel design for batched clients	Ampere+ GPUs, batch ≥ 1	Library comparator for the custom batch-1 kernel.

5.2 Starting point of this project

The project began from a research brief with a ten-week roadmap and four hypotheses (H1-H4), and a repository scaffold. The scaffold provided the directory structure, configuration files, a workload generator, an adaptive-controller rule set, a statistics module and unit tests, but the serving harness, the experiment runners, the MPI router and the CUDA kernel were skeletons that raised `NotImplementedError` or contained only a naive one-thread-per-element dequantization kernel. The brief specified an 8-billion-parameter model, vLLM or TGI serving, batch sizes 1 to 64, int8 and int4 quantization, an OpenMPI multi-replica router, a CUDA dequantization kernel, and cloud deployment.

5.3 What was retained and what changed

Table 7. Deviations from the initial plan and their reasons.

Original plan	What happened	Why
Llama-3-8B fp16 as the study model	Qwen2.5-1.5B-Instruct (fp16, GPTQ-Int8, AWQ-Int4)	An 8B fp16 model needs about 16 GB; the GPU has 8 GB. The 1.5B family has official releases at all three precisions.
Adaptive controller switches batch size and quantization live	Adaptivity implemented as routing across pre-loaded heterogeneous replicas	vLLM cannot change batch cap or precision on a running server; a restart costs about 48 s (measured).
Request rates chosen a priori	Rates derived from measured saturation capacity (utilization fractions)	The first smoke run at a guessed rate saturated the GPU, so measured tail latency would have reflected queue growth rather than policy.
Throughput = tokens / arrival window	Throughput over the busy span including drain	The first metric made every policy report the offered load.
`quantization=awq`	`awq_marlin` / `gptq_marlin`	Profiling showed 'awq' runs the older kernel; server-level throughput differs 2.1×.
Kernel ceiling stated as 4×	Ceiling computed from bytes: about 3.76×	Group-128 fp16 scales and zero points add 0.03 bytes per weight.
10-week timeline	Phases without a timeline	Requested by the project owner; work ordered by dependency.
Ten-week 'adaptive wins' framing	Pre-specified hypothesis test that could and did fail	The methodology requires a falsifiable claim; the result was negative.

5.4 Previous work in this repository

Reusable pieces that predate this report and were kept, with modification, are the workload generator (Poisson arrivals, bursty schedule, lognormal lengths; extended with a bimodal mixture and an empirical-trace mode), the confidence-interval and paired-test module (extended with Holm correction, effect sizes and a bootstrap), the rule-based adaptive controller (replaced by a hysteresis-based tier policy that operates on a pool state), and the unit-test scaffolding. Everything else described in this report was implemented during the study.

6 System design and architecture

This chapter describes what was built. Design decisions are given with the constraint that forced them, because on an 8 GB consumer GPU most of them were forced rather than chosen.

6.1 Design constraints

Table 8. Constraints and their consequences.

Constraint	Consequence for the design
8 GB of GPU memory, shared with the Windows display stack	A 1.5B-parameter model family; two replicas at 0.36 of GPU memory each (0.72 total, leaving headroom for the display and for vLLM's start-up memory check).
A running vLLM server cannot change its batch cap or quantization	Adaptivity is routing across pre-loaded heterogeneous replicas; reconfiguration is measured, not used (about 48 s per restart).
One physical GPU	Replicas time-slice the device. This is a defining property of the study, not a footnote, and gets its own controlled experiment (Section 9.6).
Windows host, Linux toolchain in WSL2	All measurements run inside WSL2; GPU performance counters are gated by a Windows driver setting, which left the kernel profiling question open.
Single operator, no cluster	The cloud phase is provided as infrastructure code with a spend cap but was not executed; no multi-node number appears in this report.

6.2 Overall architecture

The figure below shows the layers. A seeded workload generator produces request streams; an open-loop client replays them through one of three front ends (a policy router in the client process, an HTTP least-outstanding baseline, or the MPI dispatcher) to a pool of resident vLLM replicas; per-request latencies and GPU power are recorded; analysis is a pure function of the recorded files.

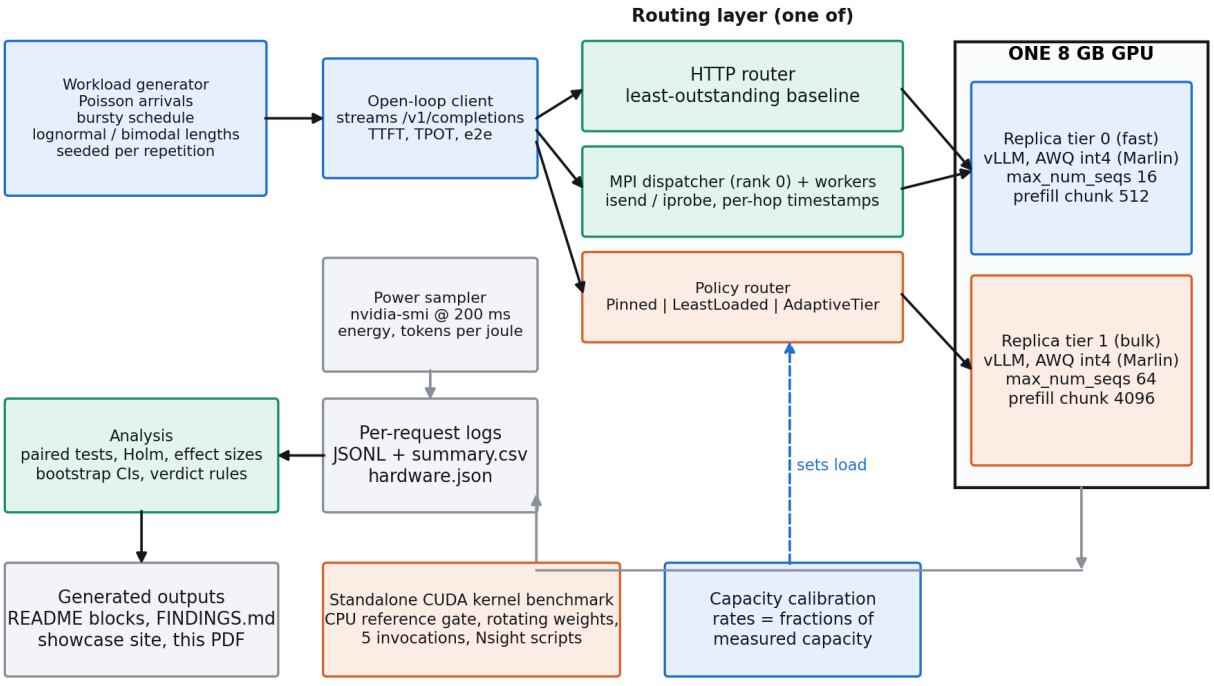


Figure 2. System architecture, from workload generation to statistical analysis.

6.2.1 Component boundaries

- **Policies are pure functions.** A routing policy receives a `PoolState` snapshot (in-flight counts, per-replica capacity, recent TTFT p95) and returns a replica index. There is no I/O in policy code, so every decision can be unit-tested and logged with a reason string.
- **Baselines share the pool.** The fixed baselines send all traffic to one replica, but both replicas stay loaded and idle replicas hold their memory. The adaptive policy therefore cannot win by being handed a better memory situation than the baseline.
- **MPI carries control and results, not tensors.** Point-to-point messages only; the routing and message-protocol logic has no MPI import and is tested without MPI.
- **The kernel is standalone.** The benchmark needs only nvcc and cuBLAS, so it builds anywhere CUDA does and can be profiled without a Python stack in the way.
- **Analysis reads files.** Everything after an experiment is a pure function of per-request records and a summary table, so results are re-derivable and this report is generated, not typed.

6.3 The replica pool

Table 9. The two-tier pool used by every routing experiment (Seqs = max_num_seqs, Batched tokens = max_num_batched_tokens, GPU mem. = gpu_memory_utilization).

Tier	Name	Weights	Seqs	Batched tokens	GPU mem.	Intended role
0	fast	AWQ-Int4 (awq_marlin)	16	512	0.36	Low-latency decode; small prefill chunks so long prompts cannot stall running decodes for long.
1	bulk	AWQ-Int4 (awq_marlin)	64	4096	0.36	Throughput and long prefill; large chunks give fast prefill at the price of higher time-per-output-token jitter.

Both tiers use the same int4 weights so that the only differences are the two scheduler parameters that the chunked-prefill literature identifies as the main throughput-latency knobs [9,19]. Making the tiers differ in precision as well would confound routing with quantization effects.

6.4 The adaptive controller

The controller is a three-level, hysteresis-based state machine chosen to be interpretable and cheap. Its only mutable state is the level.

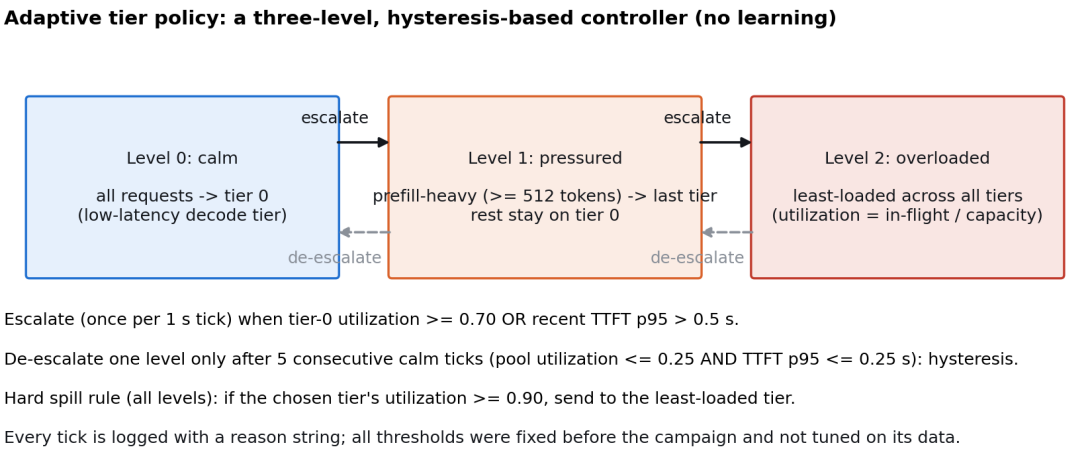


Figure 3. Controller levels and transitions. Escalation is immediate; de-escalation requires sustained calm.

Table 10. Routing rule by controller level.

Level	Name	Routing rule
0	calm	Every request goes to tier 0.
1	pressured	Requests with at least 512 prompt tokens go to the last tier (isolating long prefills); others stay on tier 0.
2	overloaded	Every request goes to the least-loaded replica by in-flight count divided by capacity.

Once per tick (1 s) the controller **escalates** one level if tier-0 utilization is at least 0.70 or the recent TTFT p95 exceeds the 0.5 s SLO. It **de-escalates** one level only after five consecutive calm ticks, where calm means pool utilization at most 0.25 and TTFT p95 at most half the SLO. A hard *spill* rule redirects a request to the least-loaded replica if its chosen replica is at 0.90 utilization or higher. The thresholds were fixed before the campaign and are not tuned to its results (Section 7.5).

The other policies are trivial by design: **fixed-fast** and **fixed-bulk** pin all traffic to one tier; **static-lb** routes each request to the least-utilized replica with no state and no length awareness.

6.5 Serving harness and client

The harness launches each vLLM replica as a separate OpenAI-compatible server process [19], waits for readiness, and records cold-start time. The client is an open-loop asynchronous HTTP client using streaming responses. For each request it records the scheduled arrival time, the send time, the time of the first streamed token, the completion time and the number of output tokens. Output length is pinned with `ignore_eos` so that all policies do identical work per request. GPU power is sampled from `nvidia-smi` every 200 ms in a separate process.

6.6 The MPI router

The router is a small MPI program [26,27] with one *dispatcher* at rank 0 and *workers* at ranks 1 to N. The dispatcher receives requests, applies the routing policy, and sends each job to a worker with a non-blocking send. Each worker owns one replica connection on an asyncio loop and polls for messages with a probe every 200 µs, so it can multiplex many outstanding requests; results return to rank 0. Every hop appends a timestamp from a monotonic clock. All ranks run on one host, where the monotonic clock is shared, so per-hop differences are exact rather than estimated (Section 7.7).

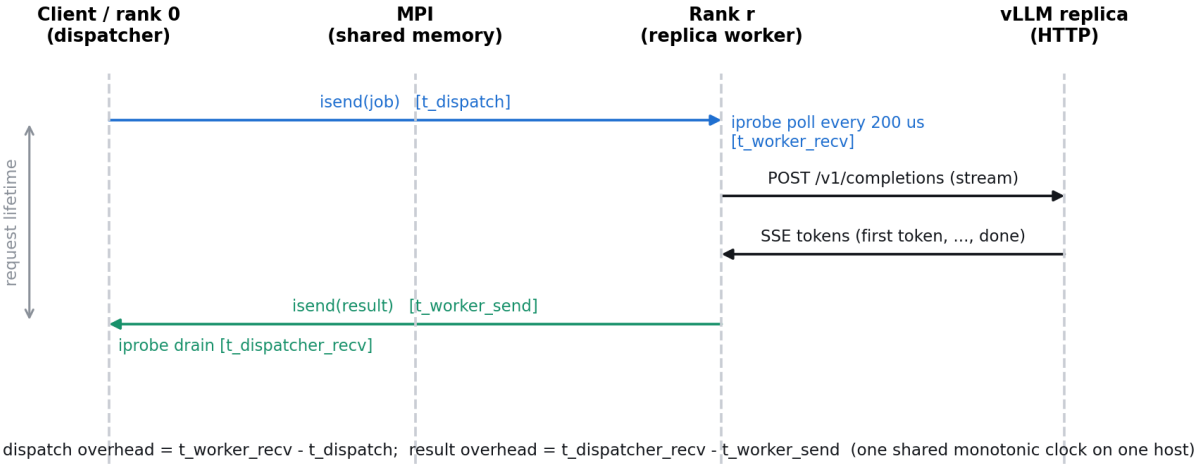


Figure 4. Request path through the MPI router with the timestamps recorded at each hop.

An HTTP router with a least-outstanding policy is provided as the comparison, so that the question ‘does MPI help’ has a control. Open MPI 5 requires `--map-by :OVERSUBSCRIBE` when ranks outnumber cores, which the microbenchmarks use.

6.7 The fused int4 kernel

6.7.1 Weight format

Weights are asymmetric group-wise int4 with group size $G = 128$, the layout that AWQ and GPTQ checkpoints use. A weight is reconstructed as $w = s(q - z)$ with $q \in [0,15]$. Eight nibbles are packed into each 32-bit word (nibble j of word w holds $k = 8w + j$), stored row-major as `uint32[N][K/8]`, with fp16 scales and zero points as `[N][K/G]`.

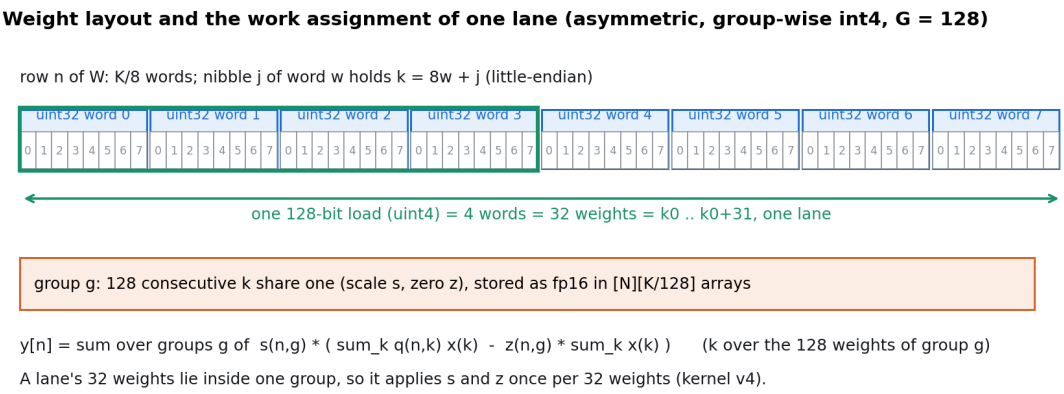


Figure 5. Packed weight layout. Consecutive k values share a 32-bit word; each group of 128 shares one scale and one zero point.

6.7.2 The algebraic form

For row n, substituting the dequantization into the dot product and factoring the group constants gives

$$y_n = \sum_g s_{n,g} \left(\sum_{k \in g} q_{n,k} x_k \; - \; z_{n,g} \sum_{k \in g} x_k \right) \tag{4}$$

so the inner loop multiplies raw nibbles by activations and the zero-point correction reuses a sum of activations that is the same for every row. The final implementation accumulates that activation sum inside the main loop at the granularity of the chunk each lane processes (Section 7.4 explains why an earlier group-level version was wrong). Versions v5 and v6 keep the same algebra.

6.7.3 Integration into vLLM

To measure the kernel where it matters, it was patched into vLLM 0.11 (`src/inferno/kernels/vllm_int4.py`). The kernel is built as a shared library with a C entry point and called through `ctypes` on PyTorch's current stream; it is registered as a torch custom op with a fake implementation so that CUDA-graph capture can record it. At model load, the AutoAWQ tensors that vLLM holds (four-bit values packed along the output axis in an interleaved order) are repacked once into the kernel's layout. The AWQ linear method is patched so that a decode step of at most a configurable number of tokens uses the custom kernel and everything else, including prefill, falls back to vLLM's own path [19]. Nothing else in vLLM changes.

6.7.4 The optimization ladder

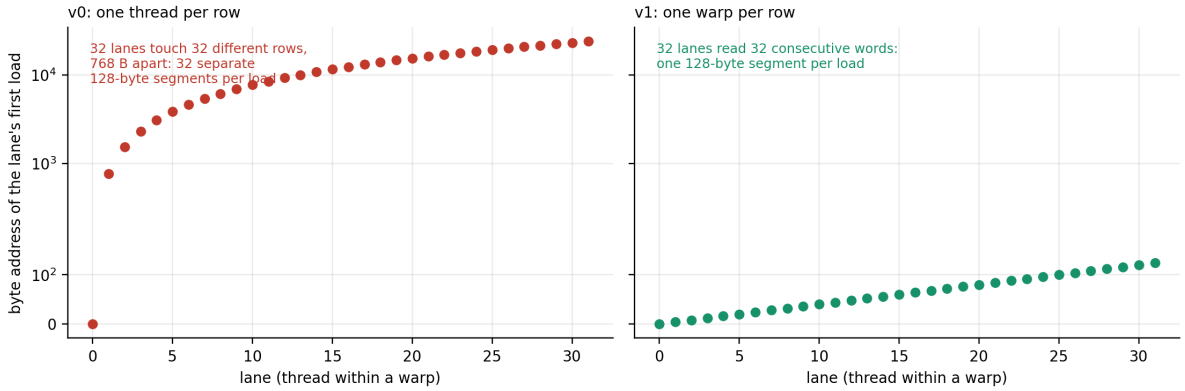


Figure 6. Memory access pattern of the thread-per-row kernel (v0) versus the warp-per-row kernel (v1).

Table 11. The kernel optimization ladder. Each step changes one thing.

Version	Change	Motivation
v0	One thread per row	Baseline; each lane strides through its own row, so a warp's loads are uncoalesced.
v1	One warp per row; 32-bit loads; shuffle reduction	Adjacent lanes read adjacent words: coalesced.
v2	128-bit loads; activations staged in shared memory (half precision)	Fewer, wider load instructions; but lane stride creates a predicted 16-way shared-memory bank conflict.
v3	v2 with one pad word per 32 activations	Shifts lanes across banks and removes the conflict.
v4	Algebraic form; streaming (`__ldcs`) loads; two loads in flight	Cut per-weight instructions; avoid polluting L2 with data read once; more memory-level parallelism.
v5	Register reuse: each 32-weight chunk decoded once into registers; activations read through the read-only cache (no shared-memory staging)	Removes the per-block staging phase and the shared-memory footprint; the same code serves B = 1..16 activation vectors.
v6	v5 with 16 lanes per row (two rows per warp)	K = 1536 has 48 chunks per row: 32 lanes leave half with two chunks and half with one; 16 lanes give every lane exactly three.

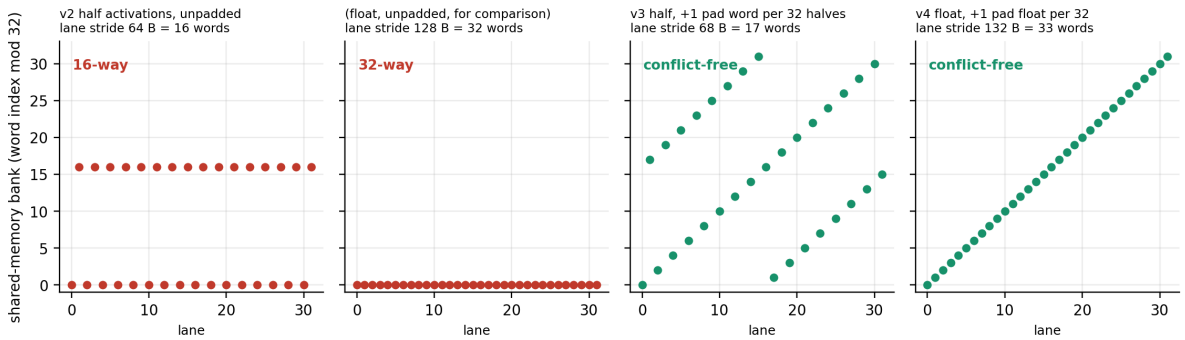


Figure 7. Shared-memory bank conflict before and after padding (predicted; the profiler counter that would confirm it was not obtainable, Section 11.2). Versions v5 and v6 no longer stage activations in shared memory.

Design source: shared-memory banking, coalescing and warp-shuffle practice follow the NVIDIA programming and best-practices guides [40]. The 16-way figure is an analytical prediction from the access pattern; this report does not claim that a profiler confirmed it.

7 Methodology

This chapter is the core of the report. It states every protocol decision, the reason for it, and what it protects against. The organizing principle is that a comparison is only as credible as the weakest thing that could differ between its arms.

7.1 Principles

1. **Pre-specification.** Hypotheses, metrics, decision rules and policy thresholds are fixed before the confirmatory data are collected, and the verdict text is produced by code from the data, not written by the author.
2. **Strongest available baseline.** The comparator for the adaptive policy is the best fixed configuration *chosen on the tested metric*, in the same pool, not a weak default.
3. **Pairing.** Repetition k uses byte-identical request streams for all policies, so differences are within-stream.
4. **Randomized order.** The order in which policies run inside each repetition is shuffled with a recorded seed to cancel drift.
5. **Instrument checked before results are trusted.** Kernels pass a correctness gate before timing; the measurement code has unit tests; the metric code has unit tests with hand-computed expected values.
6. **Report negative results.** If the pre-specified hypothesis fails, it is reported as failed, with a mechanism tested by a separate controlled experiment.
7. **No hand-typed numbers.** Every number in this report is read from a raw result file by the scripts that build it.

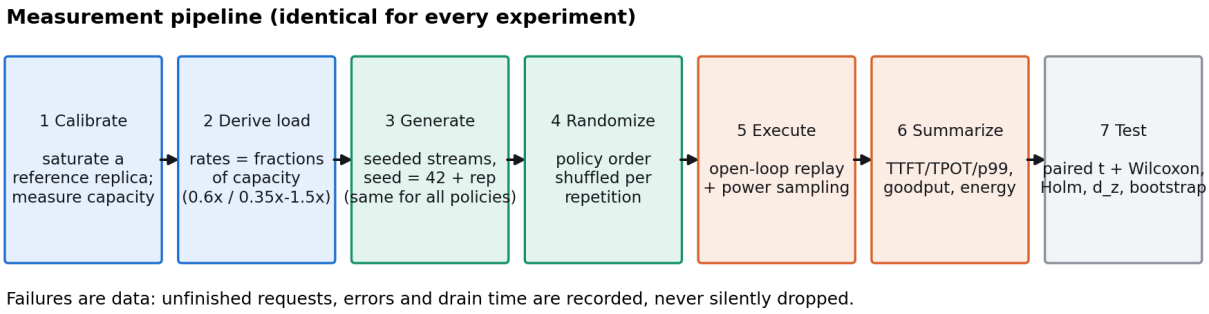


Figure 8. Measurement and analysis pipeline. Arrows are file boundaries, so each stage can be re-run independently.

7.2 Workload model and load calibration

7.2.1 Arrival processes

All experiments use an **open-loop** client: a request is issued at its scheduled time regardless of whether earlier requests have finished, and its latency is measured from the scheduled time. This follows Schroeder et al. [15]: a closed-loop client slows its own arrivals as latency grows and so hides exactly the queueing this study is about. Three patterns are used:

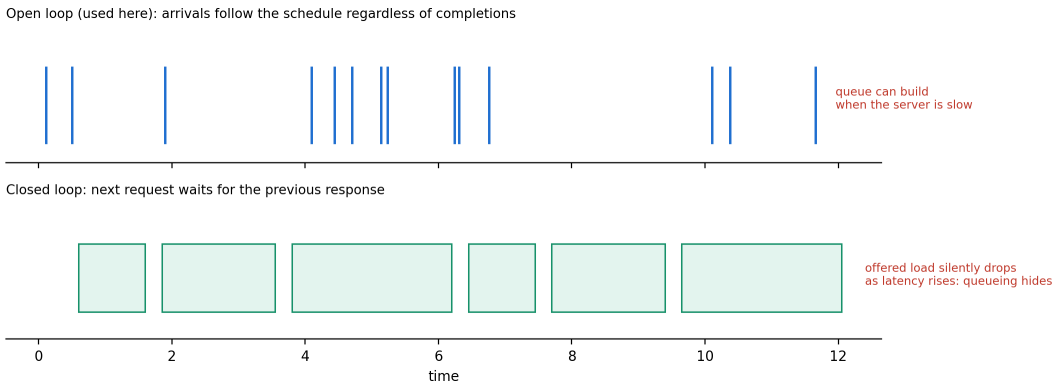


Figure 9. Open-loop versus closed-loop load generation. Under overload only the open-loop client exposes queue growth.

Table 12. Load patterns. Rates are set by calibration (below).

Pattern	Arrival process	Request lengths
steady	Poisson at a constant rate	Lognormal prompt (median 128 tokens, sigma 0.8) and output (median 256, sigma 0.6), capped at 1536 prompt and 512 output tokens.
bursty	Poisson at a base rate, with a burst of higher rate for 10 s every 40 s	As steady.
mixed-length	Poisson at a constant rate	Two-component mixture: 85% short (prompt median 96, output median 128) and 15% long (prompt median 900, output median 384).

Prompt text is synthetic and produced to a target token count; the study does not measure model output quality, and output length is fixed with `ignore_eos`. Section 7.9 discusses what that does and does not license.

7.2.2 Capacity calibration

A guessed request rate is a hidden free parameter: the first smoke run at a guessed rate saturated the GPU, so tail latency measured queue growth rather than routing quality. Rates are therefore *derived* from measured capacity. The calibration script drives one reference replica (the bulk tier) open-loop at a rate well above what it can serve, counts the output tokens completed inside a window that starts after warm-up, and converts tokens per second to requests per second using the mean output length of the length profile (computed from 20,000 sampled lengths after caps). Every load pattern is then a fixed fraction ρ of that capacity:

Table 13. Calibrated arrival rates, generated from the calibration file.

Pattern	Measured capacity (req/s)	Rule	Resulting rate (req/s)
steady	10.07	0.60 × capacity	6.043
bursty (base)	10.07	0.35 × capacity	3.525
bursty (burst)	10.07	1.50 × capacity	15.108
mixed-length	13.78	0.60 × its own capacity	8.270

The bursty pattern deliberately exceeds measured capacity during bursts, since burst absorption is the case adaptive routing is supposed to help. Steady and mixed-length load sit at 60% of one replica’s capacity, where a well-tuned fixed configuration should be adequate, which makes them the fairness check: an adaptive policy that loses there is paying overhead for nothing.

7.3 Metric definitions

Table 14. Metric definitions.

Metric	Definition	Notes
TTFT	First-token time minus scheduled arrival	Includes client-side and server-side queueing.
TPOT	(Completion time minus first-token time) / (output tokens minus 1)	Mean inter-token time for the request.
End-to-end latency	Completion time minus scheduled arrival	p50, p95, p99 over requests after warm-up.
Throughput	Completed output tokens / busy span	Busy span = max(arrival window, time of last completion). Drain time is included, so a slow configuration cannot report the offered load.
Goodput	Requests meeting both TTFT ≤ 0.5 s and TPOT ≤ 0.05 s, per second	SLO-conditioned, following DistServe.
Energy, tokens per joule	Trapezoidal integral of sampled power; tokens divided by joules	200 ms sampling; whole-GPU power, including the idle replica.
Warm-up exclusion	Requests scheduled in the first 10 s are dropped	Plus a pre-run warm-up burst.

$$\text{Goodput} = \frac{1}{T}\sum_i \mathbf{1}[\text{TTFT}_i \le 0.5\,\text{s} \ \wedge \ \text{TPOT}_i \le 0.05\,\text{s}] \tag{5}$$

Percentiles use linear interpolation (NumPy default). Tail percentiles come from few samples per run (a 75 s run at a few requests per second yields hundreds of requests), so p99 is noisy within a run; the study therefore aggregates across eight repetitions and reports confidence intervals rather than reading any single run. This is the practice recommended for tail metrics [14,31].

> **Two metric bugs found and fixed**
> Throughput was originally tokens divided by the arrival window, which made every policy that eventually finished report the offered load. A related early bug let an asyncio timer wake slightly early on Windows and produce negative TTFT; the client now loops until the scheduled time. Both are covered by regression tests (Chapter 8).

7.4 Kernel methodology

7.4.1 Roofline framing and the byte-ratio ceiling

At batch size 1 the matrix-vector product is memory-bound [3], so speedup over the fp16 path is limited by the ratio of bytes that must cross DRAM, not by arithmetic. Counting weights, scales, zero points, the input vector and the output vector, with fp16 scales and zeros and group size G:

$$\text{ceiling} = \frac{2NK + 2K + 2N}{NK/2 + 4NK/G + 2K + 2N} \quad (G = 128: \ \approx 3.76\times) \tag{6}$$

The commonly quoted ‘4×’ ignores the scale and zero-point bytes (0.0312 bytes per weight at G = 128). Reporting speedup *relative to this per-shape ceiling* is stricter and more informative than reporting it against 4×. An earlier draft of this project used 4×; it was corrected after the calculation above (Section 7.10).

7.4.2 Correctness gate

Every kernel on every shape is checked against a double-precision CPU reference on the same quantized data before any timing is recorded. The gate is a relative error below 5×10^{-3} (the output is fp16, whose rounding alone is about 5×10^{-4}). A kernel that fails is reported as INCORRECT and its time is not used. This gate caught the first version of v4, which subtracted the zero-point term of a whole 128-weight group once per 32-weight chunk and so over-corrected by roughly four times; the reported relative error was about 2 to 3.

7.4.3 Timing discipline

- **Defeating the cache.** The benchmark allocates several copies of the weights and rotates through them, so total footprint is at least max(768 MB, 24 × L2) and every launch streams from DRAM (L2 is 32 MB on this GPU). Without this, small layers would be served from L2 and would report bandwidth above the DRAM peak. The first version used 192 MB (6 × L2); a sweep of the footprint showed that this let the L2 serve a few percent of the traffic, which inflated every bandwidth figure (Section 7.10, Table in Section 9.1).
- **Achievable bandwidth.** The theoretical peak is computed from the reported memory clock and bus width. A read-only streaming kernel over a 512 MB buffer is timed as well (252 GB/s, 98% of theoretical), and results are reported against both denominators.
- **Back-to-back launches.** Times are per-launch averages over batches of launches, with median, 10th and 90th percentile.
- **Run-to-run repeat.** The whole benchmark is invoked 5 independent times; the reported time is the median of run medians, and the *spread* (max minus min over median) is reported next to it. Layers where fewer than four of the runs agree within 2% are flagged and not used for conclusions.
- **Effective bandwidth** counts only bytes that must cross DRAM and is compared with the theoretical peak (256 GB/s). Comparisons between kernels moving different byte counts use time, not bandwidth.
- **Runs within 2%.** Alongside the spread, the number of the runs whose median lies within 2% of the reported median is stored. A single outlier run (a laptop clock state) inflates the spread but leaves the median stable; the count distinguishes that case from genuine instability.
- **Shapes.** Four layer shapes taken from Qwen2.5-1.5B (QKV, output projection, gate/up, down) and two from Llama-7B, chosen because the first four are what the serving experiments run and the last two are large enough to be bandwidth-dominated.

7.4.4 In-situ comparison against the vLLM kernels

An isolated microbenchmark can mislead about what a serving stack actually runs. The three int4 paths were therefore timed *in situ*: a decode of the AWQ Qwen2.5-1.5B model in vLLM 0.11 was profiled with `torch.profiler`, kernels were grouped by layer role, and kernel-time medians over the decode steps were compared. The three paths are vLLM's Marlin kernel [6,19], its classic AWQ GEMM kernel, and the custom kernel patched into the AWQ layers, all in the same setup. Every path was measured under *both* engine modes: eager (kernels launched one by one from Python, the GPU idling between launches) and CUDA-graph decode (the GPU stays busy, as in real serving). Measuring both is a check, and it mattered (Section 7.10). Because single processes differ by up to about 10% (GPU clock state), every path and mode is profiled in three fresh processes and the per-role time is the median across processes, with the range kept.

7.4.5 End-to-end decode benchmark

Kernel time is not token time. The end-to-end benchmark launches each configuration (fp16; AWQ with Marlin; AWQ with the classic kernel; AWQ with the custom kernel for decode steps of at most 8 tokens; and a variant limited to 2 tokens) in a *fresh process*, with CUDA-graph decode, and measures output tokens per second while 1, 2, 4 and 8 sequences decode 256 tokens each (`ignore_eos`, greedy sampling). Each cell is the median of five timed repetitions inside a process; the process itself is repeated three times in a different configuration order each time, and reported values are means of those medians with a t-based interval and the paired ratio to the reference configuration. With three process repetitions the interval is wide and no p-value is computed. Two checks accompany it: each patched layer of the loaded checkpoint is compared with vLLM's own dequantize-and-matmul on random input, and the greedy token stream of the custom configuration is compared with the stock ones. The engine runs in-process (no separate engine process) so that the patch applies to it.

7.4.6 Profiler counters

The plan called for Nsight Compute counters to explain the differences between kernel versions [41]. On this machine the profiler fails with `ERR_NVGPUCTRPERM` until a Windows driver setting is changed by an administrator. The setting was not changed and no counter values appear in this report. Explanations of bank conflicts and of the slowdown of v4 on some shapes are therefore stated as hypotheses, and the alternatives that were tested and rejected are documented in Chapter 9.

7.5 Campaign design: adaptive versus fixed

7.5.1 Factors, cells and repetitions

The confirmatory campaign is a full factorial of four **policies** (fixed-fast, fixed-bulk, static-lb, adaptive) and three **load patterns** (steady, bursty, mixed-length), with eight **repetitions**: 4 × 3 × 8 = 96 runs of 75 s each, of which the first 10 s are excluded. All replicas of the pool are loaded for every policy.

repetition (seed)	fixed-fast	fixed-bulk	static-lb	adaptive
rep 0 seed 42	run 1	run 3	run 2	run 4
rep 1 seed 43	run 4	run 2	run 3	run 1
rep 2 seed 44	run 1	run 3	run 4	run 2
rep 3 seed 45	run 4	run 1	run 2	run 3
rep 4 seed 46	run 1	run 4	run 3	run 2
rep 5 seed 47	run 3	run 4	run 2	run 1
rep 6 seed 48	run 2	run 4	run 3	run 1
rep 7 seed 49	run 1	run 2	run 3	run 4

Each row replays one identical seeded request stream against every policy (pairs); the number is that policy's random position within the repetition.

Figure 10. Paired design: within a repetition all policies see the identical seeded request stream, in a randomized order.

7.5.2 Pairing and randomization

The seed for repetition k is $42 + k$, and it fixes arrival times, prompt lengths and output lengths. Every policy replays the same stream within a repetition, so the paired difference removes between-stream variability, which is large for tail latency. The order of the four policies within each repetition is a seeded random permutation, so slow effects (laptop GPU thermal state, background load) do not align with any policy.

7.5.3 Choice of baseline

The hypothesis compares the adaptive policy against the *best fixed* configuration for each load pattern. The best fixed configuration is chosen among fixed-fast, fixed-bulk and static-lb on the same metric being tested (p99), among configurations with zero request errors. Choosing the baseline after seeing the data on the tested metric makes the test conservative: it gives the baseline every chance.

7.5.4 Statistical protocol

- Headline values are means with 95% t-based confidence intervals over the eight repetitions.
- The adaptive-versus-best-fixed comparison is **paired** per repetition. Two tests are computed: the paired Student *t*-test [35] and the Wilcoxon signed-rank test [36]. A difference counts as significant only if **both** agree, and the Wilcoxon *p* is Holm-adjusted [37] across the three load patterns.
- Effect size is Cohen's paired d_z [38]; a bootstrap confidence interval of the paired difference is also computed [39].
- A **throughput non-inferiority** guard requires the adaptive policy to reach at least 98% of the best fixed throughput; otherwise a latency win is not counted, since it could be bought by shedding capacity.
- The verdict for each pattern is generated by code and has an outcome for 'adaptive better', 'no significant difference' and 'adaptive worse'.

7.5.5 Why eight repetitions

The exact Wilcoxon signed-rank test with n pairs cannot produce a two-sided p smaller than $2/2^n$, however large the effect. With Holm correction over three patterns, the smallest attainable adjusted p is three times that. Requiring the smallest attainable adjusted p to fall below 0.05 fixes the minimum sample:

Table 15. Attainable significance of the Wilcoxon test by number of pairs (computed).

Pairs n	Smallest two-sided p	Smallest Holm-adjusted p (3 patterns)	Can reach 0.05?
4	0.1250	0.3750	no
5	0.0625	0.1875	no
6	0.0312	0.0938	no
7	0.0156	0.0469	yes
8	0.0078	0.0234	yes

Seven pairs is therefore the theoretical minimum, and only just: the adjusted p would be 0.047, so a single tie or a single discordant pair would remove significance. Eight pairs (adjusted 0.023) was chosen for headroom. Studies with fewer than seven pairs (for example the five-repetition contention experiment) are labelled as unable to reach conventional significance, and effect sizes and intervals are reported instead of *p*-values.

7.5.6 Decision rule for H_1

H_1 is confirmed for a load pattern only if adaptive p99 is lower than the best fixed p99, both tests agree at Holm-adjusted 0.05, and the throughput guard holds. It is *rejected* if the difference is significant in the opposite direction. Otherwise it is undetermined. The controller thresholds (Section 6.4) were fixed before the campaign began; no threshold was adjusted after seeing campaign results.

7.6 Mechanism study: co-residency contention

The campaign result was negative in a direction that required explanation. A candidate mechanism, that two servers on one GPU interfere, was formulated as a hypothesis and tested with a separate controlled experiment rather than asserted. It is **explanatory, not confirmatory**: it was designed after seeing campaign data and is labelled that way.

Table 16. Contention study conditions.

Condition	What runs	What it isolates
single	One replica loaded; all traffic to it.	Reference.
split-idle	Both replicas loaded; all traffic to one, the other idle.	Cost of a resident but idle second server (memory split, batch caps).
split-active	Both replicas loaded; traffic split across both at the same total rate.	Cost of a second *active* server: GPU time-slicing and smaller per-server batches.

Total offered load is identical in all conditions and is expressed as a fraction ρ of capacity (ρ = 0.3 and 0.6). The comparison split-idle against single isolates residency; split-active against split-idle isolates activity. Five repetitions per cell were used, so only effect sizes and paired ratios are interpreted.

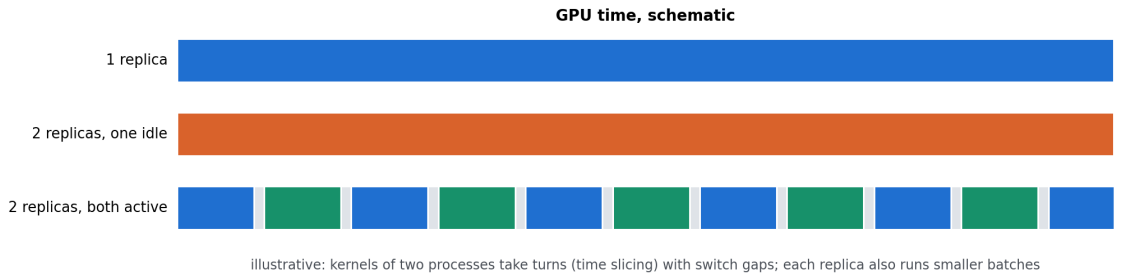


Figure 11. Why two active servers on one GPU can hurt: time-slicing between processes and smaller per-server batches.

7.7 MPI router methodology

7.7.1 Microbenchmarks

Two microbenchmarks characterize the messaging layer independent of any model. **Ping-pong** measures one-way latency by halving the round trip for message sizes from 64 B to 256 KB. **Dispatch throughput** measures jobs per second through rank 0 with 1, 2, 4, 8 and 15 workers. This machine has 16 logical cores, so the largest configurations run oversubscribed and their numbers are labelled as such. Medians over repeated trials are reported.

7.7.2 Clock argument and per-hop timestamps

The router timestamps every hop with the operating system’s monotonic clock. Because all ranks run on one host, that clock is shared, and differences between ranks are meaningful without clock-offset estimation. This argument would not hold across machines; a multi-node extension would need a clock-synchronization step, which is why cross-node claims are not made.

7.7.3 Scaling and router comparison

The scaling experiment measures throughput with 1 and 2 replicas behind (a) the HTTP least-outstanding router and (b) the MPI router, with paired repetitions. Because replicas share one GPU, this experiment measures *coordination and contention overhead*, not the scaling that independent GPUs would give. Efficiency is throughput divided by the replica count times single-replica throughput. The in-situ MPI overhead is measured from the recorded hop timestamps.

7.8 Configuration sweep and drain handling

The sweep crosses quantization (fp16, int8, int4) with batch caps (1, 8, 32, 64) at one steady load, with three repetitions per cell, and measures latency, throughput, memory, power and cold-start time. Two protocol decisions matter. First, configurations with a low batch cap cannot sustain the offered load; if such a run were allowed to drain indefinitely it would stall the sweep, so requests still unfinished 60 s after the last arrival are cancelled and counted as failures, and each cell reports its completed fraction. Second, latency percentiles of a cell that cannot finish its work are not comparable with those of a cell that can, so the results table shows the completed fraction beside latency and the discussion does not rank incomplete cells by latency.

7.9 Threats to validity

Table 17. Threats to validity.

Threat	Type	Mitigation and residual risk
Single GPU shared by all replicas	External / construct	Deliberately studied (contention experiment). Conclusions about routing do **not** extend to one-GPU-per-replica deployments; that is the main scope limit.
Small model (1.5B parameters)	External	Constrained by 8 GB. Kernel shapes from a 7B model are included in the kernel study only.
Synthetic prompts and length distributions	External	Lognormal and bimodal lengths approximate real traffic; an empirical-trace mode exists but a public trace was not used. Prefix-sharing patterns of real traffic are not represented.
Output length pinned with ignore_eos	Construct	Makes work identical across policies; removes variability in length that real requests have.
Quantized quality not evaluated	Construct	Speed and energy are reported, not accuracy. No claim is made that int4 is quality-equivalent for a task.
Laptop GPU thermal and power state	Internal	Randomized policy order per repetition; GPU clocks were not locked, so residual drift is possible.
Run length (75 s) and n = 8	Statistical	Sufficient for large effects; small effects (a few percent) would not be detected. Tail estimates are noisy per run.
Windows host with WSL2	Internal	GPU scheduling under WSL2 may differ from native Linux; not measured.
Post-hoc mechanism study	Statistical	Labelled explanatory; five repetitions; effect sizes are large so the qualitative conclusion is robust, the exact ratio is not.
Library kernel comparison at kernel level only	Construct	In-situ kernel time excludes launch gaps; the server-level check is the capacity comparison.
Unverified counters and untested cloud phase	Coverage	Reported as open items rather than filled with estimates.

7.10 Audit trail: errors found by running the protocol

The value of a protocol is measured partly by what it caught. The following defects were found during the work by checks that the protocol required; each changed a result or a claim, and each is fixed and covered by a test or by a generator.

Table 18. Defects found by the protocol and their resolution.

Defect	How it was found	Effect and fix
v4 kernel wrong (relative error 2-3)	CPU-reference correctness gate	Speed numbers for v4 would have been meaningless; fixed by accumulating activation sums per chunk.
Throughput equalled offered load	Sanity check against known saturating case	Metric redefined over the busy span including drain.
Guessed request rates saturated the GPU	First smoke run	Capacity calibration introduced.
Negative TTFT on Windows	Range check on recorded latencies	Client loops until scheduled time.
`quantization=awq` ran a different kernel than assumed	In-situ profiling	Serving pinned to awq_marlin / gptq_marlin; both paths measured and reported.
Ceiling quoted as 4×	Recomputing bytes from the layout	Ceiling computed per shape (about 3.76×); all text and figures regenerated.
Bank conflict stated as 32-way	Re-deriving the access pattern for half-precision activations	Corrected to 16-way in code comments, docs and generated text.
Early campaign numbers read as a 3× loss	Interim looks	Not reported as a result; tested by the contention experiment first.
Minimum repetitions stated as eight	Tabulating attainable p-values while writing this report	Seven is the theoretical minimum (Holm-adjusted 0.047, no margin); eight was kept for headroom and the repository documents were corrected.
Kernel benchmark rotated only 192 MB of weights	A rotation-size sweep, prompted by a kernel exceeding 100% of theoretical bandwidth	A non-LRU L2 kept part of the working set and made a 23 MB kernel about 6% too fast. Default raised to 768 MB (24 x L2); every kernel number regenerated. Earlier bandwidth percentages were inflated by that amount.
In-situ Marlin timing about twice too slow	Cross-check against end-to-end tokens per second, then re-profiling in both engine modes	The claim that the custom kernel beat Marlin on all four shapes was wrong and is retracted: Marlin is at the bandwidth limit at batch 1. The invalid file was retired; all three paths were re-measured together. The cause of the earlier value could not be established.
Test that ran a Linux binary on Windows	Test suite on the Windows host	Skipped off Linux.

8 Implementation and instrument verification

A result is only as reliable as the instrument that produced it. This chapter summarizes the implementation and, more importantly, how each instrument was verified before its output was used.

8.1 Code base

Table 19. Size of the code base (counted when this report was built).

Component	Path	Files	Lines
CUDA kernel and benchmark	`src/inferno/kernels`	6	1,114
Python package	`src/inferno`	40	4,042
Tests	`tests`	24	1,211
Scripts	`scripts`	34	5,971
Showcase site	`demo`	3	817

The `.cu/.cuh` count includes the kernel header, the standalone benchmark and an earlier naive dequantization kernel that predates this study. The `scripts` line includes this report's own build modules.

8.2 Reproducibility of the environment

- **Date-pinned dependency resolution.** An unpinned `vllm` install resolved to a PyTorch build for CUDA 13, which the installed driver (CUDA 12.7) cannot run. Resolution is now bounded by an exclude-newer date, and the versions used are recorded in `hardware.json` next to every run.
- **Environment doctor.** A check script confirms GPU visibility, CUDA toolchain, Open MPI, and package versions before any experiment.
- **Storage.** The WSL virtual disk was relocated to a larger drive after the system drive filled and produced read-only-filesystem errors mid-run.
- **Recorded provenance.** Every run directory stores the hardware and package snapshot and the exact configuration used.

8.3 Test suite

The repository contains 82 test functions in 24 files (the last full run in WSL2 collected 81 test cases, because some are parametrized, and all passed), including a test that builds and runs the CUDA benchmark; on the Windows host the GPU test is skipped, because it needs a Linux binary. Tests are grouped by what they protect:

Table 20. What the test suite covers.

Area	What the tests check
Metrics	Derived latencies; TPOT undefined for single-token outputs; warm-up and error exclusion; goodput counts only requests meeting both SLO bounds; throughput includes drain time; empty run does not crash.
Workload	Increasing arrival times; positive lengths; more requests during burst windows; reproducibility under the same seed; bimodal mixed-length profile; empirical-trace sampling.
Client	Streamed tokens and TTFT recorded; fallback to chunk counting; HTTP errors recorded, not raised; open-loop pacing; prompt length; drain timeout cancels stragglers and records failures.
Routing policies, controller and live router	Pinned policy uses only its replica; adaptive escalates under a burst and uses both tiers; static balancer spreads a burst; escalation under overload and de-escalation under light load; conservative cold-start default.
MPI routing	Least-outstanding balancing and tie-breaking; idle replica preferred; overhead split in microseconds; HTTP baseline balances across replicas.
Statistics and aggregation	Confidence intervals; paired test detects a real difference and reports none when there is none; best fixed ignores runs with errors; a clear win, a null result and a regression are each reported as such; Holm adjustment monotone and bounded.
Calibration and reporting	Capacity counts only completions inside the window; derived rates are fractions of capacity with bursts above it; mean output length reflects the mixture; generated report contains tables, figures and the H_1 verdict and states a null result plainly.
Power and hardware	Trapezoidal energy integration; too few samples handled; live sampling; multi-GPU ticks summed; hardware snapshot has the core fields.
Kernel	Builds the benchmark and runs it against the CPU reference (Linux only).

What the tests do not do

They verify the software against expected behaviour; they do not validate that the experiment measures what it claims. That second job is done by the protocol in Chapter 7 (correctness gate, calibration, paired design).

8.4 Instrument verification steps performed

Table 21. How each measuring instrument was verified.

Instrument	Verification
Kernel	Double-precision CPU reference on every shape in every benchmark invocation; correctness flag stored per kernel; timing withheld on failure.
Kernel timer	L2-defeating weight rotation; back-to-back launches; run-to-run spread reported beside every time.
Latency client	Mock-server tests; range check on recorded latencies (this found the negative TTFT).
Throughput metric	Regression test with a run whose arrivals stop early and completions trail; the metric must not equal the offered rate.
Power sampling	Unit tests on parsing and integration; number of samples per run stored and available for inspection.
MPI timestamps	All ranks on one host share the monotonic clock; a unit test checks that the overhead is split into dispatch and result components in microseconds.
Report generation	Tests build reports and check the verdict text for a win, a null result and a regression.

9 Results

All tables in this chapter are generated from the raw files under `docs/results`. Values are means with 95% confidence intervals unless stated. The chapter reports outcomes; interpretation is in Chapter 10.

9.1 Kernel study

9.1.1 Correctness

All kernels on all 6 shapes matched the CPU reference in every one of 5 benchmark invocations (no failures); the worst relative error recorded is 4.35e-04, which is consistent with fp16 output rounding.

9.1.2 Measurement corrections

Two reference measurements accompany every result. A read-only streaming kernel over 512 MB reaches 252 GB/s, 98.3% of the 256 GB/s computed from the reported memory clock; results are given against both. And the weight-rotation footprint of the benchmark was corrected. The first version rotated through 192 MB and reported a bandwidth above the theoretical peak for one kernel, which prompted a sweep: a larger footprint slowed the same kernel by about 6% while leaving cuBLAS unchanged, so the smaller footprint had let the L2 serve part of the traffic. All kernel results in this report use 768 MB.

Table 22. Effect of the weight-rotation footprint on the v5 kernel (Llama-7B MLP shape) and on cuBLAS.

rotation footprint	v5, llama7B_mlp_up (us; 3 runs)	cuBLAS fp16 (us; median of 3)
192 MB	92.9, 93.0, 93.0	360.4
384 MB	93.2, 93.2, 93.3	361.0
768 MB	98.2, 98.9, 99.0	360.9
1536 MB	98.4, 98.3, 98.3	360.9

9.1.3 Timing by version

Table 23. Median kernel time in microseconds over 5 invocations (lower is better).

layer	N x K	cuBLAS fp16	unfused	v0	v1	v2	v3	v4	v5	v6
qwen1.5B qkv	2048x1536	28.9	61.5	86.7	14.6	14.6	14.8	14.7	14.5	13.9
qwen1.5B o proj	1536x1536	24.1	44.9	81.7	13.7	14.1	13.8	13.9	12.0	11.2
qwen1.5B gate up	17920x1536	221.7	615.0	202.3	70.1	75.1	68.2	79.9	64.8	61.5
qwen1.5B down	1536x8960	113.4	269.0	704.4	41.9	42.0	39.6	48.2	33.5	33.0
llama7B attn proj	4096x4096	136.5	350.4	486.3	45.2	46.3	41.9	41.5	38.9	38.9
llama7B mlp up	11008x4096	361.3	1042.4	971.0	110.1	113.2	101.3	101.2	98.3	98.5

Table 24. Best custom kernel per shape versus fp16 cuBLAS, the byte-ratio ceiling and the two bandwidth references.

layer	best	best / cuBLAS	byte-ratio ceiling	% of ceiling	% of theoretical BW	% of streaming BW	runs within 2%
qwen1.5B qkv	v6_lane16	2.08x	3.75x	55%	47%	48%	3/5
qwen1.5B o proj	v6_lane16	2.16x	3.75x	58%	44%	45%	4/5
qwen1.5B gate up	v6_lane16	3.61x	3.76x	96%	93%	95%	5/5
qwen1.5B down	v6_lane16	3.43x	3.76x	91%	87%	88%	5/5
llama7B attn proj	v6_lane16	3.51x	3.76x	93%	90%	91%	4/5
llama7B mlp up	v5_reuse	3.67x	3.76x	98%	95%	97%	5/5

Table 25. Run-to-run spread ((max-min)/median of the run medians) and, in parentheses, how many of the 5 runs lie within 2% of the median.

layer	cuBLAS	v1	v2	v3	v4	v5	v6	correct in all runs
qwen1.5B qkv	2% (5/5)	35% (3/5)	47% (3/5)	12% (3/5)	55% (2/5)	42% (1/5)	74% (3/5)	yes
qwen1.5B o proj	3% (5/5)	4% (4/5)	13% (3/5)	5% (4/5)	5% (3/5)	9% (3/5)	5% (4/5)	yes
qwen1.5B gate up	0% (5/5)	0% (5/5)	0% (5/5)	0% (5/5)	0% (5/5)	1% (5/5)	1% (5/5)	yes
qwen1.5B down	0% (5/5)	12% (4/5)	1% (5/5)	1% (5/5)	0% (5/5)	0% (5/5)	0% (5/5)	yes
llama7B attn proj	0% (5/5)	2% (5/5)	2% (5/5)	4% (4/5)	11% (3/5)	14% (4/5)	16% (4/5)	yes
llama7B mlp up	0% (5/5)	0% (5/5)	0% (5/5)	0% (5/5)	0% (5/5)	1% (5/5)	0% (5/5)	yes

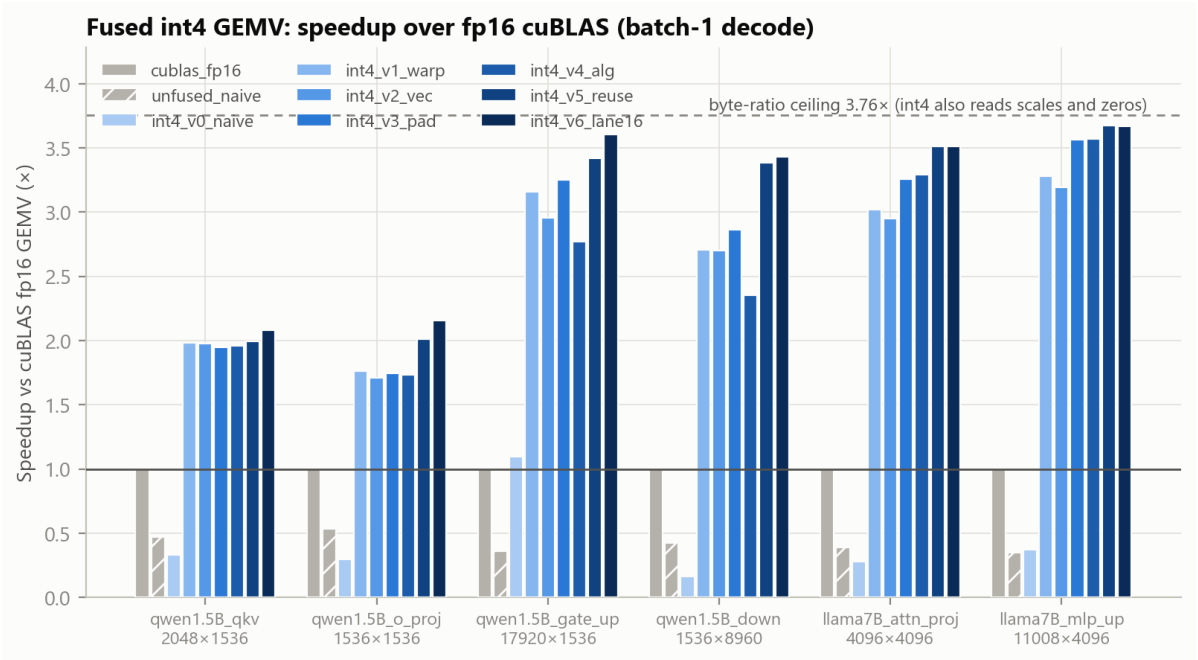


Figure 12. Speedup over fp16 cuBLAS by kernel version and layer; the dashed line is the per-shape byte-ratio ceiling.

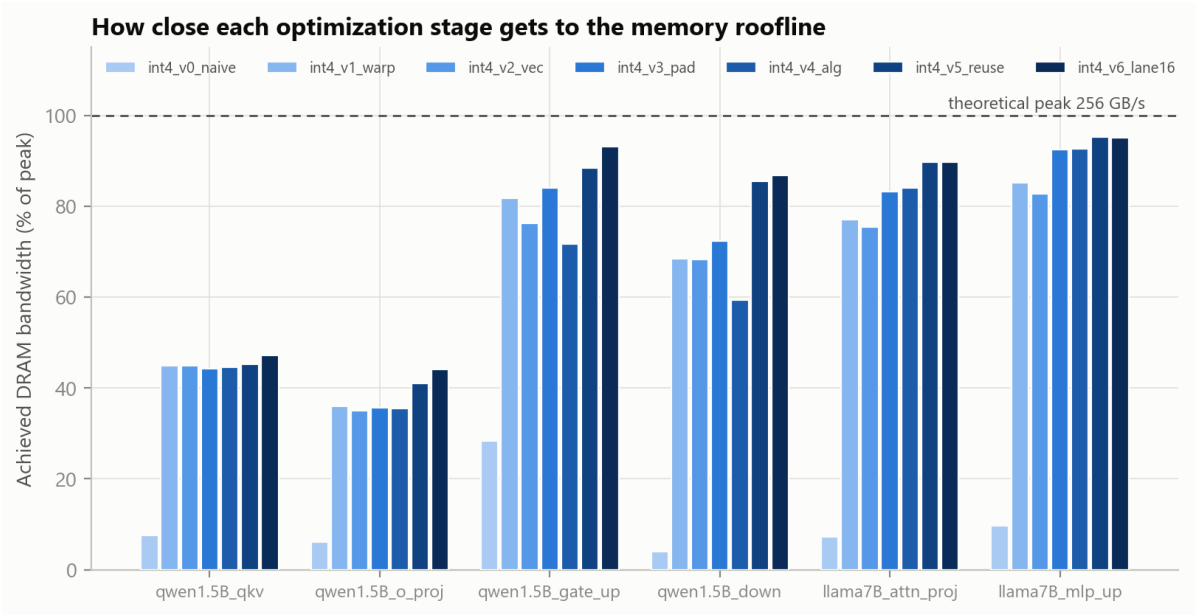


Figure 13. Achieved DRAM bandwidth as a fraction of the theoretical peak.

The best result is **llama7B mlp up** (11008×4096): 98.3 µs, 244 GB/s, 95.3% of the theoretical and 97% of the streaming-read bandwidth, and 3.67× faster than cuBLAS fp16 (361.3 µs, 98% of peak), against a byte-ratio ceiling of 3.76×. The two small layers (1 to 2 MB of weights, about 12 to 15 µs) are dominated by launch and clock jitter, and no ranking is claimed for them. On the large layers at least four of five runs agree within 2%.

9.1.4 What each optimization stage bought

Table 26. Effect of each stage on the shapes where at least 4 of 5 runs agree within 2% for every version (adjacent-version ratios of medians).

shape	v0 to v1 (coalescing)	v1 to v2 (wide loads + smem)	v2 to v3 (padding)	v3 to v4 (algebraic)	v4 to v5 (register reuse)	v5 to v6 (16 lanes)
qwen1.5B gate up	2.88× faster	1.07× slower	1.10× faster	1.17× slower	1.23× faster	1.05× faster
qwen1.5B down	16.83× faster	1.00× slower	1.06× faster	1.22× slower	1.44× faster	1.01× faster
llama7B mlp up	8.82× faster	1.03× slower	1.12× faster	1.00× faster	1.03× faster	1.00× slower

The ladder is not monotone. Widening the loads and staging activations in shared memory (v2) made the kernel *slower* than v1 on the large layers, and removing the predicted bank conflict with padding (v3) recovered it. The algebraic rewrite (v4) helped on the Llama K = 4096 shape and hurt on the Qwen shapes. Three explanations for the v4 slowdown were tested and rejected (streaming loads on short rows, a per-block prologue, late issue of scale loads). Two later changes removed it: dropping shared-memory staging altogether (v5: activations read through the read-only cache, each weight chunk decoded once into registers) and giving every lane the same number of chunks on 48-chunk rows (v6). That is consistent with the two remaining candidate explanations (the staging phase and the lane imbalance), but no profiler counter has confirmed which one is responsible. Loading several chunks per lane before consuming them never helped once lanes were balanced.

9.1.5 Small batches

Table 27. Speedup of the custom kernel over fp16 cuBLAS (same B) for B activation vectors; median of 3 invocations, all results matched the CPU reference.

B	qwen1.5B gate up	qwen1.5B down	llama7B mlp up	byte-ratio ceiling (first shape)
1	3.60x	3.45x	3.67x	3.76x
2	3.37x	3.23x	3.48x	3.75x
3	2.34x	1.90x	2.40x	3.74x
4	1.80x	1.45x	1.83x	3.74x
6	1.16x	1.17x	1.41x	3.72x
8	1.22x	0.92x	1.03x	3.71x
12	0.83x	0.62x	0.60x	3.68x
16	0.48x	0.69x	0.46x	3.65x

The kernel keeps a clear advantage through B of about 4 to 6, is roughly at break-even around B of 7 to 8, and is slower than fp16 cuBLAS beyond that. Per weight it costs about three instructions per batch element on CUDA cores, so it leaves the memory-bound regime early; a tensor-core kernel is needed for larger batches. The result is therefore a batch-1 and very-small-batch result (for example speculative decoding), not a general replacement.

9.2 Comparison with vLLM’s kernels, end-to-end decode and server capacity

9.2.1 Kernel time in situ

Table 28. Custom kernel versus the vLLM kernels, in situ (CUDA-graph decode, kernel time in microseconds, median of 3 fresh processes; ratio below 1 means the custom kernel is faster).

layer	shape	custom standalone (us)	custom in situ (us)	vLLM Marlin (us)	vLLM AWQ (us)	custom / Marlin	custom / AWQ
qkv	2048x1536	13.9	10.2	10.3	12.2	1.00x	0.84x
o_proj	1536x1536	11.2	8.0	8.4	11.4	0.96x	0.71x
gate_up	17920x1536	61.5	69.9	60.1	72.3	1.16x	0.97x
down	1536x8960	33.0	37.1	31.8	41.9	1.17x	0.88x

Table 29. The same kernels under both engine modes (in-situ median of 3 fresh processes, microseconds).

path	mode	qkv	o_proj	gate_up	down	sum per layer
vLLM Marlin	graphs	10.3	8.4	60.1	31.8	110.5
vLLM classic AWQ	graphs	12.2	11.4	72.3	41.9	137.8
custom kernel (v6)	graphs	10.2	8.0	69.9	37.1	125.2
vLLM Marlin	eager	11.4	9.2	68.9	36.3	125.7
vLLM classic AWQ	eager	12.4	12.3	64.8	40.8	130.3
custom kernel (v6)	eager	9.1	7.3	60.5	32.2	108.9

In CUDA-graph decode, the custom kernel takes between 0.96 and 1.17 of Marlin’s time on the four layer shapes (a ratio above 1 means slower) and between 0.71 and 0.97 of the classic AWQ

kernel's. Marlin is at the bandwidth limit on the large layers at batch 1. **An earlier version of this report claimed the custom kernel beat Marlin on all four shapes; that claim was wrong.** It rested on a Marlin timing about twice as slow as the values above, which could not be reproduced: end-to-end throughput disagreed with it, and re-profiling Marlin in the same session under both engine modes gave the current values. The cause of the earlier number was not established, and the file was retired. One plausible cause: 2 of the 18 profiling processes run for this report had every kernel, FlashAttention included, running 2 to 5 times slower than in their sibling processes (a GPU-wide slowdown, for example throttling), and a single-process measurement taken during such an episode would look exactly like the retracted one; this could not be verified for the old run. Each in-situ time is therefore the median of three fresh processes, and even among the remaining processes the largest process-to-process spread was 39% (classic AWQ, o_proj eager).

9.2.2 End-to-end decode throughput

Table 30. Decode throughput in output tokens per second (mean of three process repetitions with 95% interval; parentheses: paired ratio to vLLM's default Marlin path).

configuration	1 seq	2 seqs	4 seqs	8 seqs
fp16 (no quantization)	67 ± 0 (0.47x)	132 ± 2 (0.47x)	263 ± 3 (0.48x)	515 ± 11 (0.49x)
AWQ-int4, vLLM classic kernel	136 ± 4 (0.95x)	264 ± 8 (0.94x)	516 ± 8 (0.94x)	968 ± 36 (0.91x)
AWQ-int4, vLLM Marlin (default)	143 ± 2	280 ± 5	547 ± 7	1060 ± 6
AWQ-int4, custom kernel (<= 2 tokens)	143 ± 2 (1.00x)	268 ± 11 (0.96x)	512 ± 18 (0.94x)	954 ± 68 (0.90x)
AWQ-int4, custom kernel (<= 8 tokens)	144 ± 2 (1.01x)	272 ± 7 (0.97x)	370 ± 2 (0.68x)	472 ± 6 (0.44x)

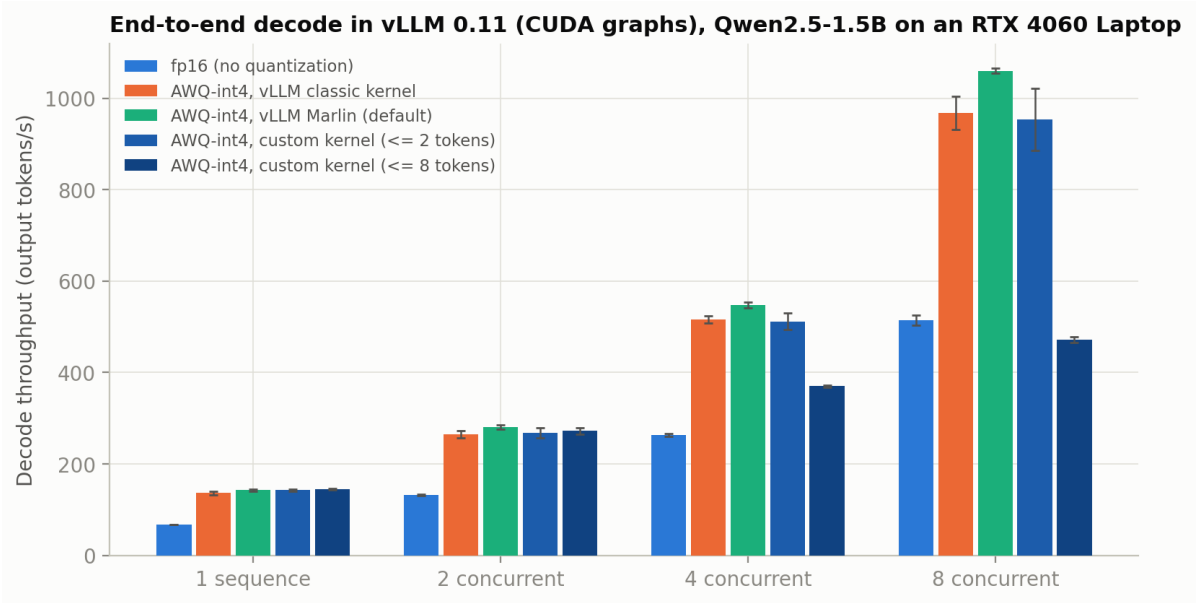


Figure 14. End-to-end decode throughput in vLLM 0.11 with CUDA graphs.

With one sequence the custom kernel decodes at 144 tokens/s against 143 for Marlin (paired ratio 1.01, range 1.00 to 1.02), 136 for the classic AWQ kernel and 67 for fp16. That is parity with Marlin, an improvement over the classic kernel, and about 2.1× fp16. With four concurrent sequences the paired ratio to Marlin is 0.68: Marlin's tensor-core kernel is faster once several sequences decode together. A hybrid that uses the custom kernel only for steps of at most two tokens follows the same pattern. All 112 patched layers of the loaded checkpoint matched vLLM's own dequantize-and-matmul to a maximum relative error of 1.1e-03. Greedy text agreement on the first prompt: custom b8 vs awq_classic: 256 of 256 tokens identical; custom

b8 vs awq_marlin: 256 of 256 tokens identical; custom b2 vs awq_classic: 256 of 256 tokens identical; custom b2 vs awq_marlin: 256 of 256 tokens identical; awq_classic vs awq_marlin (stock vs stock): 256 of 256 tokens identical. Every configuration, stock or custom, produced the identical 256-token continuation. This is one prompt, so it is a sanity check on the integration and not an accuracy evaluation.

9.2.3 Server-level capacity

Saturated with the same open-loop request process, a vLLM server using the Marlin path sustained 2828 output tokens/s against 1354 for the classic AWQ path, a ratio of 2.09×. A batch-1 kernel is a latency-sensitive, low-concurrency result and says little about saturated serving.

9.3 Where the custom kernel earns its place: every layer, batch 1 to 64

The in-situ comparison above covers batch 1 and a few concurrency levels. To decide whether the kernel belongs in the serving path at all, every Qwen2.5-1.5B decode layer was timed at batch sizes 1, 2, 4, 8, 16, 32, 64 against vLLM's Marlin path, the classic AWQ path and fp16 cuBLAS, each called exactly as vLLM calls it, on the *same* AutoAWQ weights converted once into each layout. Every cell is one CUDA graph holding one call per weight copy, with copies rotated over at least 768 MB (24× L2) so each call streams its weights from DRAM, replayed 20 times per trial over 7 trials; launch overhead is excluded for every implementation alike. A cell counts as a win only when the winner's slowest trial beats the runner-up's fastest. The decision rule was fixed before the run: carry the kernel into vLLM only if it consistently wins on layers that account for meaningful decode time.

Table 31. Custom-kernel time divided by Marlin time (below 1, in bold, means the custom kernel is faster). Share: fraction of quantized-GEMM time per decode step at batch 1. n/a: the custom kernel is instantiated only up to batch 16; the deployed path falls back above that.

Layer (N × K)	share	B=1	B=2	B=4	B=8	B=16	B=32	B=64
qkv (2048×1536)	8%	**0.92**	1.03	1.85	2.93	5.39	n/a	n/a
o_proj (1536×1536)	7%	**0.88**	**0.89**	1.71	3.08	5.00	n/a	n/a
gate_up (17920×1536)	58%	1.01	1.05	2.02	4.02	7.42	n/a	n/a
down (1536×8960)	28%	1.08	1.04	2.25	3.79	7.17	n/a	n/a

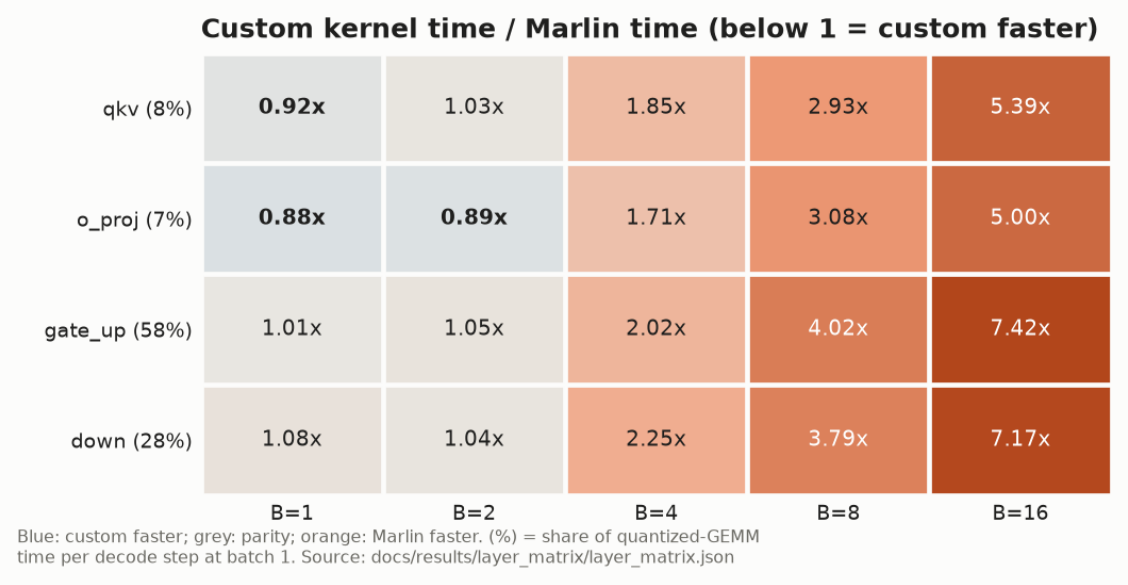


Figure 15. Where the custom kernel earns its place: custom/Marlin time ratio by layer and batch size.

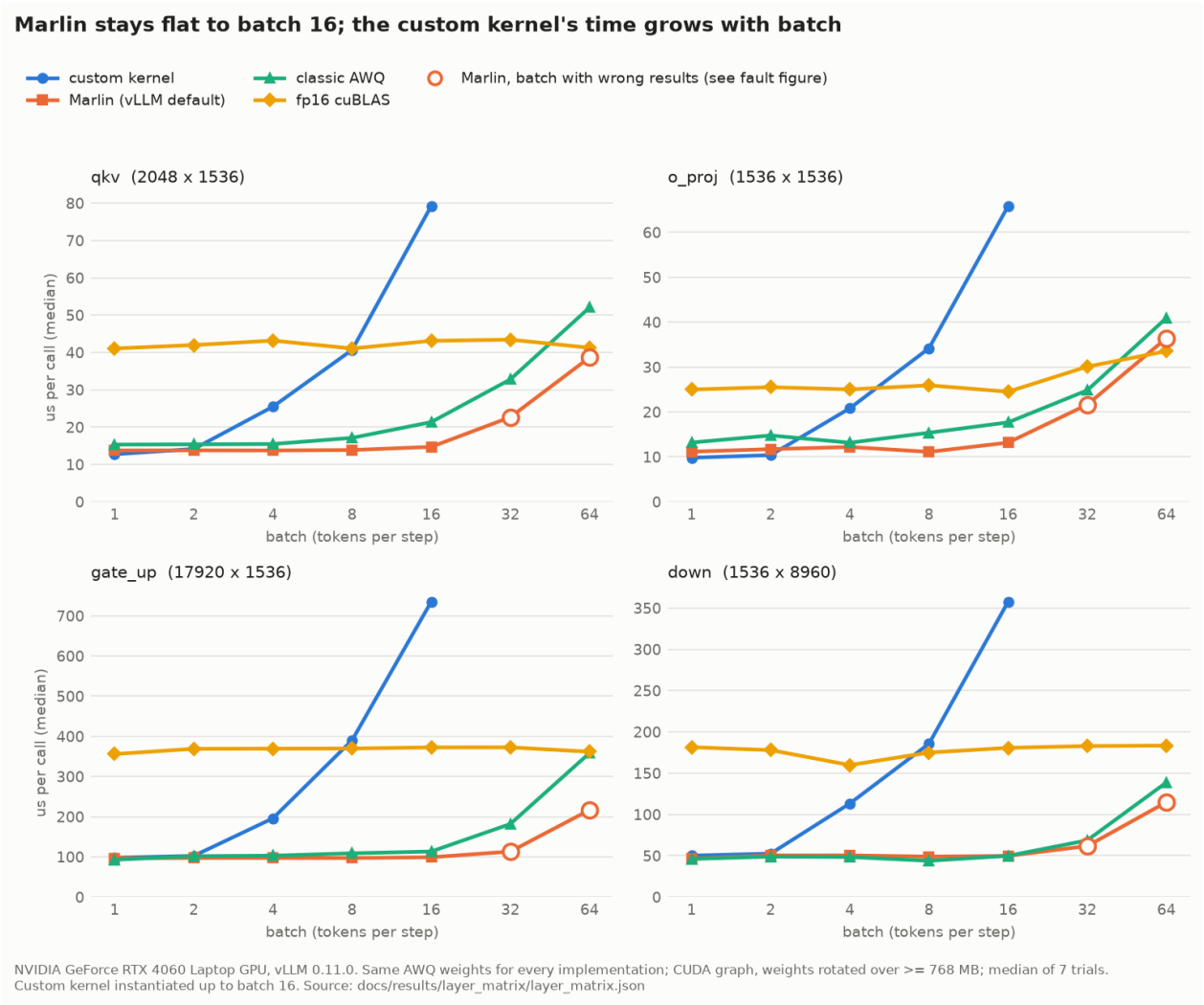


Figure 16. Microseconds per call by batch size for each Qwen2.5-1.5B decode layer and implementation. Hollow markers: batch sizes at which Marlin's output was wrong in the correctness probe (next section).

At batch 1 the custom kernel clearly wins only on qkv and o_proj (15% of decode GEMM time) and ties on the two large layers that carry 85% of it. From batch 4 it is 1.7 to 2.2× slower, rising

to 7.4× at batch 16: its time grows almost linearly with the batch (each weight is multiplied on CUDA cores once per token), while Marlin's tensor-core path stays flat to batch 16 because weight traffic, not arithmetic, sets its time. The one-time layout conversion is negligible for both (at most 10 ms per layer for the custom layout and 4 ms for Marlin's, paid once at load). Every implementation matched the dequantize-and-matmul reference (custom kernel worst 4e-04) except one Marlin cell, which led to the next section.

Decision
The built-in Marlin path already matches the custom kernel on the layers that dominate decode and is far faster at every batch above 2. Following the pre-stated rule, the kernel is not made a serving default and the paired end-to-end serving test of a modified engine is not run; the opt-in switch (`vllm_int4.install()`) remains as a research tool, and engineering effort moves to the routing experiment.

9.4 Correctness probe: Marlin returned wrong results at some batch sizes

The reference check above flagged Marlin on one layer. Marlin was then called 40 times with identical inputs per cell, in four variants: back-to-back with vLLM's shared workspace (as vLLM runs it), with NaN-filled allocations between calls, with a freshly zeroed workspace for every call, and with a device synchronize between calls. Batches 32, 64, 128 produced intermittent wrong results on every layer tested, with relative max errors up to 0.28; batches 1, 8, 16, 48 were always correct (worst 4e-04). The custom kernel, the classic AWQ kernel and fp16 are deterministic and correct on the same inputs. A freshly zeroed workspace does not remove the fault, so stale lock state in the shared workspace is not the explanation; the cause was not established.

Table 32. Marlin calls with wrong output (relative max error above 1e-2) out of 40 identical calls. plain: shared workspace as in vLLM; fresh_ws: newly zeroed workspace per call. All variants: docs/results/marlin_fault/report.md.

Layer	variant	B=1	B=8	B=16	B=32	B=48	B=64	B=128
down	plain	0	0	0	19	0	6	12
down	fresh_ws	0	0	0	40	0	40	40
gate_up	plain	0	0	0	27	0	33	14
gate_up	fresh_ws	0	0	0	34	0	10	28
qkv	plain	0	0	0	21	0	12	20
qkv	fresh_ws	0	0	0	17	0	20	16

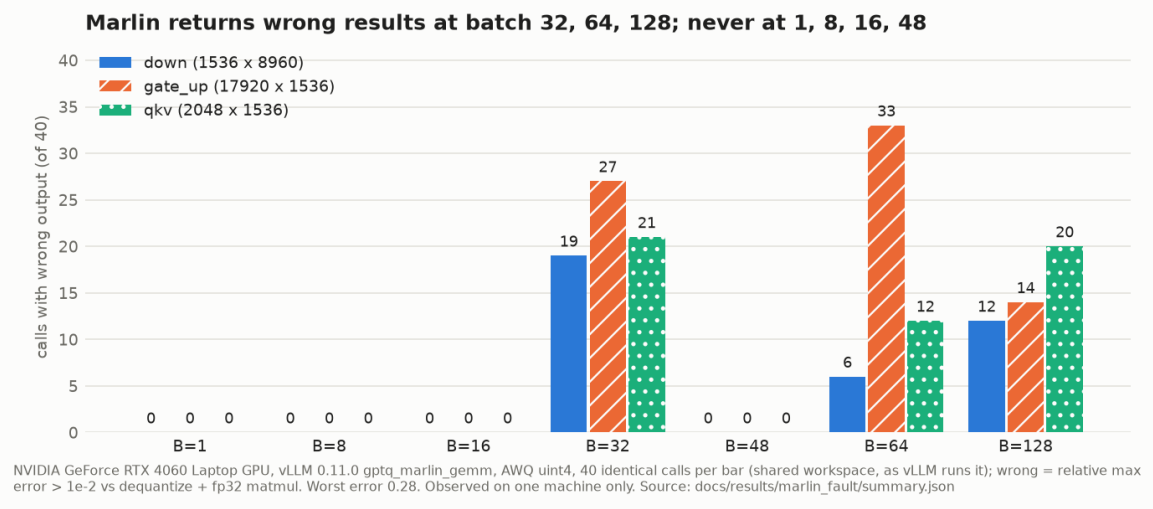


Figure 17. Marlin calls with wrong output by batch size and layer (shared workspace, 40 calls per bar).

Status: observed on one machine (WSL2, laptop GPU); not yet reproduced on other hardware. vLLM pads decode batches to captured sizes that include 32 and 64, so if the fault is general it would corrupt real serving at moderate concurrency; it may equally be specific to WSL2, this driver (566.07) or this laptop GPU. The real-checkpoint check (greedy decode of the AWQ model at batch 1 versus 32/64, against the classic AWQ path) was written but could not run on the development machine during this revision, and no second machine was available. It is therefore not reported upstream, and the Marlin timings at the affected batch sizes in the previous section carry this caveat. `make marlin-probe` and `make quick-bench` reproduce it; docs/REPRODUCE.md asks independent testers to report either outcome.

9.5 Campaign: adaptive versus fixed routing

The three tables below give, for each load pattern, the four policies over eight repetitions. Latencies are end-to-end in seconds; throughput is output tokens per second over the busy span; goodput is requests per second meeting both SLO bounds.

Table 33. Steady load. Mean ± 95% CI over 8 paired repetitions.

policy	n	p50 (s)	p95 (s)	p99 (s)	TTFT p99 (s)	throughput (tok/s)	goodput (req/s)	tokens/J
fixed-fast	8	4.22 ± 0.92	7.14 ± 1.11	7.98 ± 1.17	3.66 ± 1.12	1549 ± 28	1.41 ± 0.88	19.6 ± 0.4
fixed-bulk	8	2.68 ± 0.10	5.68 ± 0.29	6.18 ± 0.43	0.05 ± 0.01	1583 ± 38	5.66 ± 0.16	19.9 ± 0.5
static-lb	8	6.93 ± 0.32	16.54 ± 1.15	18.60 ± 1.45	0.12 ± 0.01	1488 ± 34	5.32 ± 0.15	18.3 ± 0.4
adaptive	8	6.78 ± 0.31	16.69 ± 1.23	18.24 ± 1.27	0.12 ± 0.01	1491 ± 37	5.33 ± 0.15	18.4 ± 0.5

Table 34. Bursty load. Mean ± 95% CI over 8 paired repetitions.

policy	n	p50 (s)	p95 (s)	p99 (s)	TTFT p99 (s)	throughput (tok/s)	goodput (req/s)	tokens/J
fixed-fast	8	12.30 ± 2.20	18.64 ± 1.68	20.12 ± 1.57	16.50 ± 1.67	1285 ± 42	0.10 ± 0.15	16.2 ± 0.5
fixed-bulk	8	4.06 ± 0.18	10.37 ± 0.40	11.44 ± 0.20	3.13 ± 0.87	1419 ± 45	3.81 ± 0.09	18.2 ± 0.5
static-lb	8	8.53 ± 0.35	21.75 ± 1.45	24.24 ± 1.54	7.86 ± 1.57	1348 ± 46	3.00 ± 0.11	16.5 ± 0.6
adaptive	8	8.54 ± 0.52	21.94 ± 1.16	24.29 ± 1.39	7.98 ± 1.45	1339 ± 37	2.97 ± 0.17	16.3 ± 0.4

Table 35. Mixed-length load. Mean ± 95% CI over 8 paired repetitions.

policy	n	p50 (s)	p95 (s)	p99 (s)	TTFT p99 (s)	throughput (tok/s)	goodput (req/s)	tokens/J
fixed-fast	8	2.05 ± 0.25	5.03 ± 0.20	5.93 ± 0.36	1.94 ± 0.46	1411 ± 34	4.44 ± 0.95	17.9 ± 0.4
fixed-bulk	8	1.60 ± 0.06	4.94 ± 0.16	6.13 ± 0.15	0.06 ± 0.00	1425 ± 39	7.91 ± 0.28	18.2 ± 0.5
static-lb	8	4.25 ± 0.16	13.54 ± 1.00	17.63 ± 0.83	0.13 ± 0.01	1345 ± 34	7.47 ± 0.26	16.7 ± 0.4
adaptive	8	4.20 ± 0.16	13.47 ± 0.84	18.04 ± 0.91	0.13 ± 0.01	1349 ± 37	7.49 ± 0.26	16.8 ± 0.4

Table 36. Pre-specified test H_1: adaptive versus the best fixed configuration (paired, Holm-corrected).

load pattern	best fixed	n	adaptive p99 (s)	best-fixed p99 (s)	change	Holm p	d_z	thr. ratio
bursty	fixed-bulk	8	24.29	11.44	+112%	0.023	8.2	0.94x
mixed length	fixed-fast	8	18.04	5.93	+204%	0.023	12.0	0.96x
steady	fixed-bulk	8	18.24	6.18	+195%	0.023	10.0	0.94x

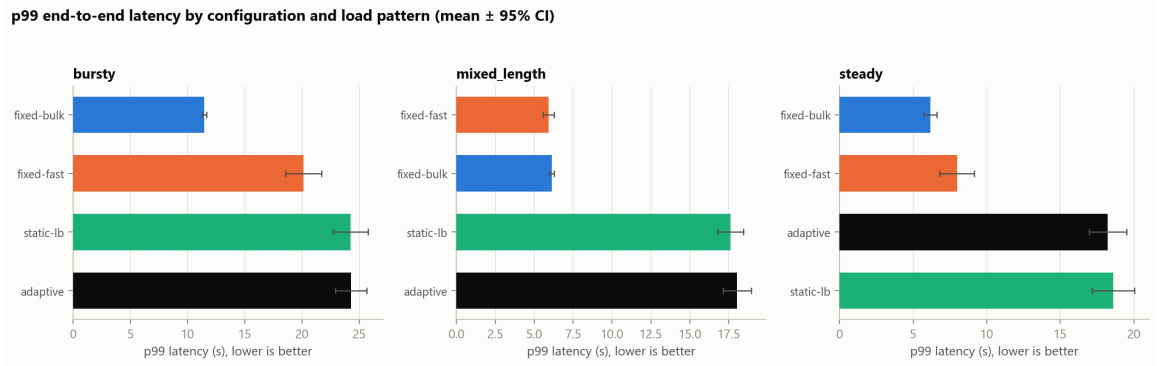


Figure 18. End-to-end p99 latency by policy and load pattern (mean and 95% CI).

In every pattern all eight pairs point the same way (the smallest Wilcoxon p attainable at $n = 8$, 0.0078, which Holm-adjusts to 0.023 across three patterns). The adaptive policy is significantly **worse** than the best fixed configuration on p99 in all three patterns, and its throughput is between 94% and 96% of the best fixed policy's, so it also fails the non-inferiority guard. **H_1 is rejected.**

9.5.1 Headroom analysis

Table 37. Post hoc headroom check on the campaign data: how much could any policy that chooses the better fixed tier per pattern gain?

load pattern	fixed-fast p99 (s)	fixed-bulk p99 (s)	better fixed tier	gain of per-pattern choice over fixed-bulk	adaptive vs better fixed
bursty	20.12	11.44	fixed-bulk	0.0%	+112%
mixed length	5.93	6.13	fixed-fast	3.3%	+204%
steady	7.98	6.18	fixed-bulk	0.0%	+195%

The better of the two fixed tiers differs by pattern, but the tier that is best overall (fixed-bulk) is within 3.3% of the per-pattern best on every pattern. On this shared-GPU pool a pattern-aware switch between the tiers could therefore have gained at most 3.3% of p99; the test of H_1 was close to unwinnable independent of the co-residency penalty of the policy actually tested. This analysis was done after seeing the data and is explanatory.

9.5.2 Energy and throughput

Table 38. Energy efficiency and throughput, adaptive versus the best fixed configuration for p99 (means over 8 repetitions).

load pattern	adaptive tokens/J	best-fixed tokens/J	change	adaptive throughput	best-fixed throughput	ratio
bursty	16.3	18.2	-10.0%	1339	1419	0.94
mixed length	16.8	17.9	-6.3%	1349	1411	0.96
steady	18.4	19.9	-7.9%	1491	1583	0.94

Mean fraction of requests the adaptive controller sent to tier 0 / tier 1: bursty 34% / 66%; mixed length 34% / 66%; steady 33% / 67%. Its behaviour is close to a load balancer once the controller has escalated.

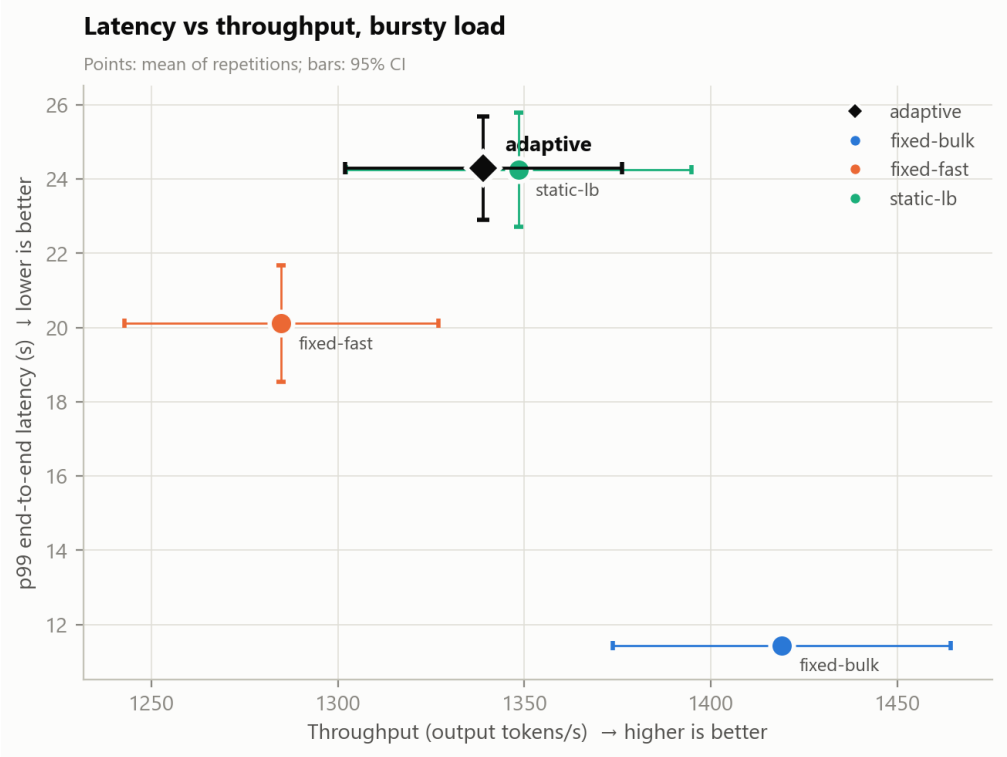


Figure 19. Latency-throughput trade-off per policy under bursty load (each point is one repetition).

9.6 Mechanism study: two active servers on one GPU

Table 39. Contention experiment cells (5 paired repetitions).

load	condition	n	p99 (s)	throughput (tok/s)	tokens/J
rho = 0.3	single	5	4.46 ± 0.13	828	10.9
rho = 0.3	split-idle	5	4.49 ± 0.15	828	10.9
rho = 0.3	split-active	5	9.25 ± 0.17	792	10.1
rho = 0.6	single	5	6.21 ± 0.60	1595	20.0
rho = 0.6	split-idle	5	6.26 ± 0.67	1595	20.1
rho = 0.6	split-active	5	13.02 ± 0.79	1505	18.2

Table 40. Paired ratios (first over second condition).

load	comparison	metric	ratio	pairs
rho = 0.3	split-idle vs single	e2e_p99	1.008	5
rho = 0.3	split-idle vs single	throughput_tok_per_s	1.000	5
rho = 0.3	split-idle vs single	tokens_per_joule	1.007	5
rho = 0.3	split-active vs split-idle	e2e_p99	2.060	5
rho = 0.3	split-active vs split-idle	throughput_tok_per_s	0.957	5
rho = 0.3	split-active vs split-idle	tokens_per_joule	0.920	5
rho = 0.6	split-idle vs single	e2e_p99	1.009	5
rho = 0.6	split-idle vs single	throughput_tok_per_s	1.000	5
rho = 0.6	split-idle vs single	tokens_per_joule	1.002	5
rho = 0.6	split-active vs split-idle	e2e_p99	2.079	5
rho = 0.6	split-active vs split-idle	throughput_tok_per_s	0.944	5
rho = 0.6	split-active vs split-idle	tokens_per_joule	0.908	5

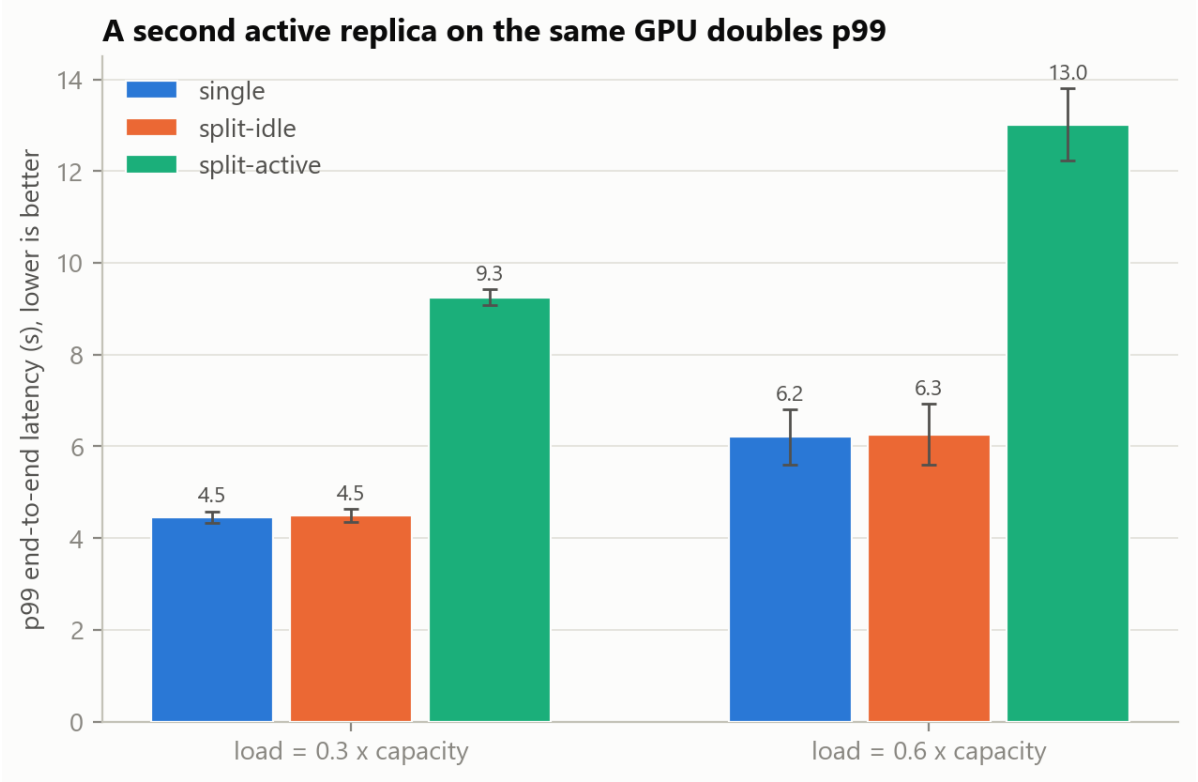


Figure 20. Contention experiment summary.

With a second replica loaded but idle, p99 differs from the single-replica reference by only +0.8% and +0.9% at the two loads. With the second replica *active* at the same total offered load, p99 rises by a factor of 2.06 and 2.08. The cost comes from activity, not from memory being split. With five pairs the Wilcoxon test cannot go below 0.0625, so the strength of this conclusion rests on the effect size and consistency across pairs, not on a *p*-value.

9.7 MPI routing

Table 41. Ping-pong latency between two ranks on one host.

message size	round trip p50 (µs)	round trip p99 (µs)	one-way estimate (µs)
64 B	3.7	12.3	1.9
512 B	4.4	24.5	2.2
4 KB	9.5	31.6	4.7
32 KB	25.5	65.5	12.7
256 KB	753.9	1347.0	376.9

Table 42. Dispatch-loop throughput through rank 0 (payload 512 B, window 64; 16 logical cores).

workers	jobs / s	round trip p50 (µs)	round trip p99 (µs)	note
1	186,314	322	609	
2	169,525	339	975	
4	161,832	337	1060	
8	123,287	420	1204	
15	49,654	765	11960	oversubscribed

One-way latency is a few microseconds for messages up to 4 KB, about 13 µs at 32 KB and a few hundred microseconds at 256 KB. The jump is steeper than bandwidth alone would explain, consistent with a switch of message protocol; the threshold was not determined. Dispatch throughput falls as workers are added, and the largest worker count, which is oversubscribed, is the slowest.

9.8 Scaling and MPI versus HTTP with real replicas

Table 43. Scaling under saturating load with all replicas on one GPU (paired repetitions).

router	replicas	n	throughput (tok/s)	p99 (s)	efficiency vs 1 replica
HTTP	1	4	2106 ± 31	74.0 ± 7.1	1.00
HTTP	2	4	1980 ± 35	82.2 ± 6.6	0.47
MPI	1	4	2118 ± 31	73.3 ± 7.2	1.00
MPI	2	4	1980 ± 29	82.5 ± 8.1	0.47

Table 44. MPI router compared with the HTTP router (both tests must agree for significance).

replicas	MPI vs HTTP throughput	paired t-test p	Wilcoxon p	pairs
1	+0.58%	0.047	0.125	4
2	+0.01%	0.945	1.000	4

Adding a second replica on the same GPU *reduces* total throughput (efficiency 0.47 of linear scaling). MPI and HTTP routing differ by less than 1% in throughput; with four pairs the Wilcoxon test cannot go below 0.125, and where the *t*-test alone is borderline the two tests disagree, so no difference is claimed. Inside the real router, the measured dispatch hop has a median of 2.0 ms, about a thousand times the microbenchmark: the worker’s single asyncio loop is also parsing streaming responses for many requests, so hop time includes queueing behind that work. Even so it is small next to multi-second request latencies at this load.

9.9 Quantization and batch-cap sweep

Table 45. Sweep at one steady load (3 repetitions per cell; unfinished requests 60 s after the last arrival counted as failures).

config	reps	throughput (tok/s)	tokens/J	p50 (s)	completed	cold start (s)
fp16-bs1	3	0 ± 0	0.0	n/a	0%	200
fp16-bs8	3	376 ± 21	5.0	61.7	44%	49
fp16-bs32	3	1385 ± 55	16.9	9.3	100%	45
fp16-bs64	3	1491 ± 68	17.9	5.5	100%	47
int8-bs1	3	0 ± 0	0.0	n/a	0%	151
int8-bs8	3	627 ± 11	8.4	46.2	72%	52
int8-bs32	3	1554 ± 68	18.8	3.6	100%	53
int8-bs64	3	1553 ± 67	18.8	3.6	100%	50
int4-bs1	3	5 ± 13	0.1	119.7	1%	49
int4-bs8	3	885 ± 17	12.5	33.8	100%	47
int4-bs32	3	1589 ± 68	19.9	2.6	100%	47
int4-bs64	3	1589 ± 69	19.9	2.6	100%	47

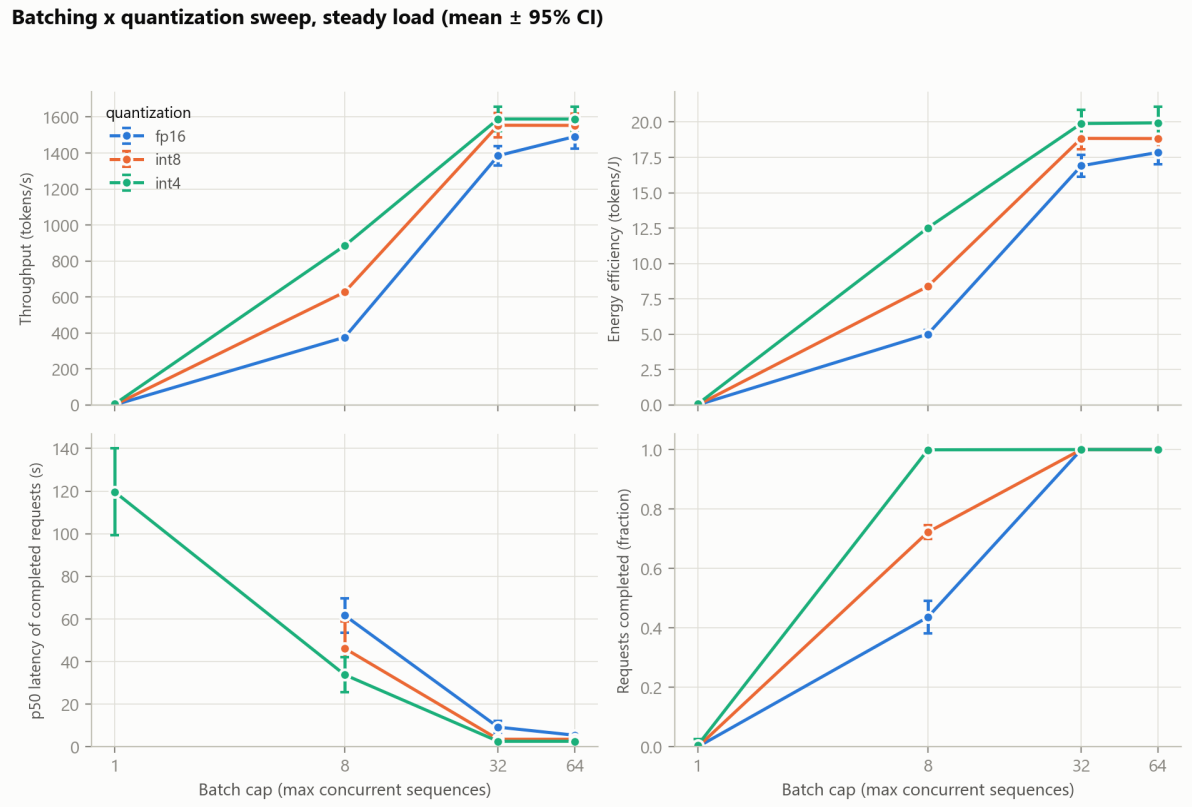


Figure 21. Sweep results: latency, throughput and energy efficiency by configuration.

Cells with batch cap 1 could not sustain the load, and some completed none of their requests inside the window, so their latency entries are not comparable. Among cells that finish, int4 lowers median latency and raises tokens per joule relative to fp16 at the same offered load; the exact figures are in the table. Restarting a server to change configuration took a median of 48 s of cold start, several times the 10 s burst duration used in the bursty pattern. Cold starts above 100 s, which occurred on the first load of the fp16 and int8 weights, are excluded from that median.

10 Discussion

10.1 Answers to the research questions

Table 46. Research questions and answers.

Question	Answer supported by the data
RQ1 Kernel vs roofline and vLLM	On the best bandwidth-bound layer the kernel reached 95% of theoretical DRAM bandwidth (97% of the measured streaming-read bandwidth) and 3.67× cuBLAS. Inside vLLM with CUDA graphs it decodes at parity with the default Marlin path at one sequence (1.01×), ahead of the classic AWQ kernel, and slower than Marlin with four sequences (0.68×). It does not beat Marlin; an earlier claim that it did was a measurement error. Above a batch of about 8 a CUDA-core kernel loses to cuBLAS. Across every Qwen layer at batch 1 to 64 it wins only qkv/o_proj at batch 1 (15% of decode GEMM time) and is up to 7.4× slower than Marlin, so Marlin stays the default.
RQ2 Adaptive vs fixed	No. Adaptive routing was significantly worse on p99 in all three patterns, with lower throughput and lower tokens per joule. H_1 is rejected.
RQ3 Mechanism	Two active replicas time-slicing one GPU roughly double p99 (2.06× and 2.08×) at fixed total load; an idle second replica costs about nothing.
RQ4 MPI	Point-to-point latency is a few microseconds; end-to-end, MPI and HTTP routing are indistinguishable in this setup, and dispatch adds about two milliseconds inside the real router. Scaling on one GPU is negative because of co-residency, not because of MPI.
RQ5 Quantization, batch cap, reconfiguration	int4 improves latency and energy per token at fixed load; small batch caps cannot sustain the load; reconfiguration costs about 48 s.

10.2 Status of the four hypotheses

Table 47. Hypothesis outcomes.

ID	Outcome	Evidence and qualifications
H1	Rejected	Significantly worse p99 in all patterns; non-inferiority guard also failed. Applies to single-GPU replicas.
H2	Partly supported	Scaling on the shared GPU is sub-linear (efficiency below 1); MPI overhead is measured (microseconds isolated, about 2 ms in situ). No replica count at which MPI overhead erodes gains was reached, because two replicas were the maximum on one GPU.
H3	Refuted as stated; explanation unresolved	The kernel reaches the memory limit on large layers rather than a fraction of the library path, and is at parity with Marlin end to end at batch 1 (not a win). The layer matrix confirms it wins only small layers at batch 1. The part of H3 that requires profiler counters to explain differences was not completed.
H4	Not supported	The adaptive policy lowered tokens per joule by a few percent. The only efficiency gain measured came from quantization, which is a model change.

10.3 Why the adaptive policy lost

1. **The routing premise assumed independent capacity.** Sending some requests to the second replica only helps if the second replica adds capacity. On one GPU it does not: the device is shared, and total useful capacity does not grow.
2. **Active co-residency is expensive.** The contention experiment shows that an active second server roughly doubles p99 at fixed load, while an idle one is free. The best fixed policies avoid this by using only one server.
3. **The adaptive policy activates the second server.** After escalation it spreads requests across both replicas, so it inherits the penalty, and it tracks the static load balancer closely.
4. **Smaller per-server batches** are a likely additional contributor: splitting the stream reduces what each server can batch. This was not separated from time-slicing in the experiment.

This is consistent with the premise of disaggregation systems that put specialized replicas on separate devices [2,13]. The result does not say that adaptive routing is a bad idea; it says the idea was tested here in the one configuration where its precondition (independent capacity per replica) fails. The prediction that follows, and which this study did not test, is that with one replica per GPU the penalty disappears and the comparison becomes open.

10.4 Why the kernel result should not be over-read

- The kernel result is **batch 1 and very small batches**, memory-bound, on one GPU. Above a batch of about 8 it loses to fp16 cuBLAS, and end to end Marlin is clearly faster from four sequences.
- **Parity with Marlin is not a win.** Marlin is at the bandwidth limit on the large layers, so there is no headroom to beat it at batch 1 on this GPU; the honest claim is that a from-scratch kernel reached the same place.
- Reaching the byte-ratio ceiling on one shape shows the kernel is near the limit for that shape, not that it is state-of-the-art for all shapes. The small layers sit far below the ceiling, consistent with launch and clock jitter dominating 12 to 15 microsecond kernels.
- The layer matrix makes this concrete: the only clear wins are the two small layers at batch 1, which carry about one seventh of the quantized-GEMM time.
- End-to-end decode also spends time outside the quantized layers (the fp16 output projection over the vocabulary, attention, normalization, sampling), so a faster int4 kernel moves tokens per second less than it moves kernel time.

10.5 Methodological observations

- Several defects in the audit trail (Section 7.10) were found only because an instrument was deliberately checked. The correctness gate, the drain-aware throughput, and calibration each changed reported results.
- The power argument mattered in practice: with fewer than seven pairs the pre-specified test could not have confirmed anything, and with seven only with no margin. The contention experiment, at five pairs, is therefore reported through effect sizes rather than *p*-values.
- Deriving loads from capacity prevented a class of results in which every policy looked bad because the offered load exceeded what the GPU could serve.
- Two of the errors overturned earlier claims: a benchmark that let the L2 serve part of the traffic (bandwidth figures were too high by about 6%), and a baseline timing that made the custom kernel look faster than Marlin. Both were caught by cross-checks (a reading above 100% of peak, and end-to-end throughput disagreeing with kernel times), not by review. A kernel comparison is only credible when it is closed at a second level of measurement.

11 Limitations and future work

11.1 Limitations

- **One GPU, one model family, one engine.** Every conclusion is conditional on an RTX 4060 Laptop GPU, Qwen2.5-1.5B and vLLM 0.11.
- **Replicas share a GPU.** This is the dominant limitation for the routing result and the reason it should not be generalized to multi-GPU deployments.
- **No profiler counters.** The bank-conflict prediction and the causes of the v4 slowdown and of the v5/v6 gains are hypotheses (the counter permission was re-checked during the work and is still blocked).
- **The end-to-end comparison is one model on one GPU** with three process repetitions per configuration, so intervals are wide and the parity with Marlin is a statement about a few percent, not about small differences.
- **The Marlin correctness fault is observed on one machine.** It is reproducible there at the kernel level, but it has not been checked on the real checkpoint end to end or on a second GPU, so it is not attributed to vLLM.
- **No cloud run.** Infrastructure code with a cost cap is provided but was not applied; no multi-node or multi-GPU result is reported.
- **Synthetic workloads and quality not evaluated.** No accuracy or perplexity comparison across quantization levels was made.
- **Modest repetition counts.** Eight repetitions for the campaign, five for the mechanism study, four for scaling, three for the sweep; small effects are out of reach.
- **Bibliography.** 16 references were verified against publisher or proceedings pages during preparation (marked †); the rest are cited from knowledge and should be confirmed before formal submission.
- **The dashboard** was checked with rendered screenshots rather than interactive sessions across browsers.

11.2 Future work, in order of expected value

1. **Enable GPU performance counters and run Nsight Compute** on the kernel ladder (one-time driver setting, then `make kernel-profile`), to confirm or refute the bank-conflict and occupancy hypotheses and to explain the v4 slowdown.
2. **Reproduce the Marlin fault elsewhere** (`make quick-bench` on another GPU, then the real-checkpoint probe). If it reproduces, report it upstream with the minimal script; if not, record it as machine-specific.
3. **Run the cloud phase** with one replica per GPU. This is the experiment that tests the prediction in Chapter 10: with independent capacity, does load- and length-aware routing beat the best fixed configuration?
4. **Try MPS** for concurrent kernel execution across the two replicas on one GPU, to see whether the co-residency penalty is reducible without extra hardware.
5. **A tensor-core variant for batches of 8 to 64**, the range where the CUDA-core kernel loses to cuBLAS and to Marlin, so that one code path covers decode at every concurrency.
6. **Extend the vLLM integration and upstream it as a discussion**: the GPTQ path, the quantized output projection (which needs an accuracy evaluation), and a benchmark issue that presents the measured parity and its limits.
7. **Real traces and quality.** Replace synthetic lengths with a published conversation trace and add a task-accuracy check for the quantized checkpoints.
8. **Additional adaptive mechanisms** such as migration of running requests, and predictive rather than reactive control.

12 Conclusion

This study built and measured an LLM-serving testbed spanning a CUDA kernel, its integration into vLLM, an MPI router and an adaptive routing controller on a single 8 GB GPU, with the protocol and statistics fixed in advance. Five findings stand. First, a hand-written fused int4 GEMV can reach the memory roofline (95% of theoretical bandwidth, 3.67× cuBLAS, against a 3.76× byte-ratio ceiling) on a large layer. Second, inside vLLM it is at parity with the default Marlin path at one sequence (144 against 143 tokens/s, ratio 1.01), ahead of the classic AWQ kernel and of fp16, and behind Marlin with four concurrent sequences (0.68); it does not beat the library. Third, the hypothesis that adaptive routing over pre-loaded replicas would beat the best fixed configuration was **rejected**: p99 was 112% to 204% worse, a controlled experiment attributes this to two active servers contending for one GPU, and the headroom for any pattern-aware choice was at most 3.3% anyway. Fourth, MPI routing is cheap in isolation and indistinguishable from HTTP routing end to end in this setup. Fifth, across every decode layer at batch 1 to 64 the kernel wins only layers worth 15% of decode GEMM time, so by the pre-stated rule Marlin stays the default; the same benchmark’s reference check exposed batch-dependent wrong results from Marlin on this machine, a finding that awaits reproduction elsewhere.

The main contribution is less any one number than the demonstration that each claim was tested with an instrument that had itself been checked, including two checks that overturned the study’s own earlier claims, and that the study reports what it did not achieve: no profiler counters, no cloud run, no win over the production kernel, no independent reproduction yet, and a routing result that applies only to replicas sharing a device.

Appendix A. Reproducing the study

The commands below are the Makefile targets used for the reported runs, run from the repository root inside WSL2 (Ubuntu) with the environment created by the setup scripts. Long targets run for tens of minutes to hours; the campaign alone is 96 runs of 75 s plus server start-up.

```
make setup           # date-pinned Python environment (uv), vLLM, PyTorch
make toolchain       # CUDA 12.4 toolchain, gcc 12, Open MPI (micromamba)
make test            # unit tests
bash scripts/repeat_kernel_bench.sh 5   # ladder benchmark, five invocations, CPU-reference gate
bash scripts/rerun_kernel_all.sh    # library + 5 ladder runs + 3 batch runs
bash scripts/flush_sweep.sh         # rotation-size check
bash scripts/insitu_graphs.sh       # in-situ kernel timing, both engine modes
bash scripts/run_e2e_matrix.sh 3    # end-to-end decode matrix (vLLM integration)
python scripts/headroom_analysis.py # post hoc routing headroom
make mpi-bench       # MPI ping-pong and dispatch microbenchmarks
make calibrate       # measure capacity, derive load rates
make sweep           # quantization x batch-cap sweep
make scaling         # MPI vs HTTP, 1 and 2 replicas
make campaign        # 4 policies x 3 patterns x 8 repetitions
make contention      # mechanism study
make report          # regenerate tables, figures, verdicts
make site            # regenerate the showcase dashboard
make kernel-profile  # Nsight Compute (requires GPU counters enabled)
```

The environment is date-pinned, so that a re-run resolves the same package versions. Results depend on the GPU model, driver, and thermal state; numbers from another machine should be compared by shape (ratios, ceiling fractions) rather than by absolute value.

Appendix B. Configuration

Replica pool and controller thresholds (`configs/pool_config.yaml`):

```
base_port: 8100
pool:
  - name: fast
    quantization: int4
    batch_size: 16
    max_num_batched_tokens: 512
    gpu_memory_utilization: 0.36
  - name: bulk
    quantization: int4
    batch_size: 64
    max_num_batched_tokens: 4096
    gpu_memory_utilization: 0.36
adaptive:
  long_prompt_tokens: 512
  escalate_util: 0.70
  deescalate_util: 0.25
  calm_ticks_required: 5
  spill_util: 0.90
  tick_sec: 1.0
```

Workload definition and derived rates for the campaign are stored with the results as `docs/results/campaign/config_snapshot.json`; the calibrated rates are tabulated in Section 7.2.

Table 48. Hardware and package snapshot recorded with the campaign (`hardware.json`).

item	value
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
driver	566.07
vllm	0.11.0
torch	2.8.0
numpy	2.2.6
httpx	0.28.1

Appendix C. Glossary

term	meaning
AWQ	Activation-aware Weight Quantization; a 4-bit weight-only quantization method.
Batch cap	Maximum number of sequences a server decodes together (`max_num_seqs`).
Byte-ratio ceiling	Ratio of bytes moved by the fp16 path to those moved by the int4 path for one matrix-vector product; the upper bound on speedup for a bandwidth-bound kernel.
Chunked prefill	Splitting a long prompt into pieces processed over several scheduler iterations.
Cohen's d_z	Paired effect size: mean of the paired differences divided by their standard deviation.
Decode	The token-by-token generation phase; memory-bound at small batch.
Goodput	Rate of requests that meet the latency service-level objective.
Holm correction	Step-down adjustment of p-values controlling the family-wise error rate.
KV cache	Stored attention keys and values of previous tokens.
Open loop	Load generation in which arrivals do not depend on completions.
Prefill	The prompt-processing phase; compute-bound for long prompts.
Roofline	Performance model bounding attainable throughput by compute and memory bandwidth roofs.
SLO	Service-level objective; here TTFT ≤ 0.5 s and TPOT ≤ 0.05 s.
TPOT	Time per output token after the first.
TTFT	Time to first token, from scheduled arrival.
Tokens per joule	Output tokens divided by GPU energy over the busy span.

Appendix D. Provenance of results

Every table in Chapters 6 to 9 that contains a measured value is generated by the modules in `scripts/report_pdf` from the following files, all under `docs/results`:

Table 49. Result files behind the tables.

Result	Source file
Kernel times, correctness, spread	kernel_bench.json (median of 5 invocations)
In-situ kernel times (both engine modes)	insitu/awq_marlin_{graphs,eager}.json, insitu/awq_{graphs,eager}.json, insitu/awq_inferno_{graphs,eager}.json
Small-batch kernel sweep	kernel_batch_runs/run_*.json
Rotation-size check	flush_sweep/*.json
End-to-end decode in vLLM	e2e_kernel/*_rep*.json, e2e_kernel/summary.json
Headroom analysis	campaign/headroom.json
Capacity and derived rates	calibration.json, calibration_awq_kernel.json
Campaign	campaign/summary.csv, campaign/summary.json, campaign/config_snapshot.json
Contention	contention/contention_summary.json
MPI microbenchmarks	mpi_microbench.json
Scaling and router comparison	scaling/scaling_summary.json
Sweep	sweep/summary.csv
Hardware and packages	campaign/hardware.json

References

References are numbered in order of first citation. Entries marked † (16 of them) were checked against the publisher, proceedings or arXiv page while preparing this report. The remaining entries are cited from the author's knowledge; venue, year and page details should be confirmed before formal submission. Software documentation is cited by name and version.

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762.

[2] Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, I., Maleki, S., Bianchini, R. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting.* ACM/IEEE ISCA 2024. arXiv:2311.18677. †

[3] Williams, S., Waterman, A., Patterson, D. *Roofline: An Insightful Visual Performance Model for Multicore Architectures.* Communications of the ACM 52(4):65-76, 2009. †

[4] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* ICLR 2023. arXiv:2210.17323. †

[5] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., Han, S. *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.* MLSys 2024 (Best Paper). †

[6] Frantar, E., Castro, R. L., Chen, J., Hoefler, T., Alistarh, D. *MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models.* ACM PPoPP 2025. arXiv:2408.11743. †

[7] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., Stoica, I. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* ACM SOSP 2023. †

[8] Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., Chun, B.-G. *Orca: A Distributed Serving System for Transformer-Based Generative Models.* USENIX OSDI 2022. †

[9] Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., Ramjee, R. *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve.* USENIX OSDI 2024, pp. 117-134. †

[10] Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., Lin, W. *Llumnix: Dynamic Scheduling for Large Language Model Serving.* USENIX OSDI 2024. †

[11] Crankshaw, D., Wang, X., Zhou, G., Franklin, M. J., Gonzalez, J. E., Stoica, I. *Clipper: A Low-Latency Online Prediction Serving System.* USENIX NSDI 2017, pp. 613-627. †

[12] Li, Z., Zheng, L., Zhong, Y., Liu, V., Sheng, Y., Jin, X., Huang, Y., Chen, Z., Zhang, H., Gonzalez, J. E., Stoica, I. *AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving.* USENIX OSDI 2023, pp. 663-679. †

[13] Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving.* USENIX OSDI 2024. †

[14] Dean, J., Barroso, L. A. *The Tail at Scale.* Communications of the ACM 56(2):74-80, 2013.

[15] Schroeder, B., Wierman, A., Harchol-Balter, M. *Open Versus Closed: A Cautionary Tale.* USENIX NSDI 2006. †

[16] Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., Dean, J. *Efficiently Scaling Transformer Inference.* MLSys 2023. arXiv:2211.05102.

[17] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Re, C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. arXiv:2205.14135. †

[18] Dao, T. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.* ICLR 2024. arXiv:2307.08691.

[19] vLLM project. *vLLM documentation and source* (v0.11.0): OpenAI-compatible server, quantization backends (awq, awq_marlin, gptq_marlin), scheduler parameters (max_num_seqs, max_num_batched_tokens). https://github.com/vllm-project/vllm

[20] Leviathan, Y., Kalman, M., Matias, Y. *Fast Inference from Transformers via Speculative Decoding.* ICML 2023. arXiv:2211.17192.

[21] Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., Sheng, Y. *SGLang: Efficient Execution of Structured Language Model Programs.* NeurIPS 2024. arXiv:2312.07104.

[22] Romero, F., Li, Q., Yadwadkar, N. J., Kozyrakis, C. *INFaaS: Automated Model-less Inference Serving.* USENIX ATC 2021. arXiv:1905.13348.

[23] Dettmers, T., Lewis, M., Belkada, Y., Zettlemoyer, L. *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* NeurIPS 2022. arXiv:2208.07339.

[24] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., Han, S. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* ICML 2023. arXiv:2211.10438.

[25] NVIDIA. *Multi-Process Service (MPS)* and *Multi-Instance GPU (MIG)* user guides. NVIDIA documentation.

[26] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.0.* 2021.

[27] Gabriel, E., Fagg, G. E., Bosilca, G., Angskun, T., Dongarra, J. J., Squyres, J. M., et al. *Open MPI: Goals, Concept, and Design of a Next Generation MPI Implementation.* EuroPVM/MPI 2004.

[28] Culler, D., Karp, R., Patterson, D., Sahay, A., Schauser, K. E., Santos, E., Subramonian, R., von Eicken, T. *LogP: Towards a Realistic Model of Parallel Computation.* ACM PPoPP 1993.

[29] Little, J. D. C. *A Proof for the Queuing Formula: $L = \lambda W$.* Operations Research 9(3):383-387, 1961.

[30] Georges, A., Buytaert, D., Eeckhout, L. *Statistically Rigorous Java Performance Evaluation.* ACM OOPSLA 2007, pp. 57-76. †

[31] Hoefler, T., Belli, R. *Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results.* SC 2015. †

[32] Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P. F. *Producing Wrong Data Without Doing Anything Obviously Wrong!* ACM ASPLOS 2009.

[33] Kalibera, T., Jones, R. *Rigorous Benchmarking in Reasonable Time.* ACM ISMM 2013.

[34] Reddi, V. J., Cheng, C., Kanter, D., Mattson, P., et al. *MLPerf Inference Benchmark.* ACM/IEEE ISCA 2020.

[35] Student (W. S. Gosset). *The Probable Error of a Mean.* Biometrika 6(1):1-25, 1908.

[36] Wilcoxon, F. *Individual Comparisons by Ranking Methods.* Biometrics Bulletin 1(6):80-83, 1945.

[37] Holm, S. *A Simple Sequentially Rejective Multiple Test Procedure.* Scandinavian Journal of Statistics 6(2):65-70, 1979.

[38] Cohen, J. *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum, 1988.

[39] Efron, B. *Bootstrap Methods: Another Look at the Jackknife.* Annals of Statistics 7(1):1-26, 1979.

[40] NVIDIA. *CUDA C++ Programming Guide* and *CUDA C++ Best Practices Guide* (shared memory banks, memory coalescing, warp-level primitives). NVIDIA documentation.

[41] NVIDIA. *Nsight Compute Documentation* and *NVIDIA GPU Performance Counter Permissions (ERR_NVGPUCTRPERM).* NVIDIA developer documentation.